Database Design Principles and Methodologies

a) **Conceptual Database Design**
   - The E-R Model

b) **Design principles and Methodologies**
   - Life cycle of databases within Information Systems
   - Quality of conceptual models

c) **Logical database Design**
   - Translating from the E-R model into the Relational Model

d) **Normalization**
   - Functional dependencies and normal forms

e) **Physical Database Design**
   - (hints to) Access structures, Indexing, Query plans

The database Design Process

Lifecycle

Designing a database

The design of a database is one of the steps in the **lifecycle** of any information system, which generally consists of the following activities:

- Collection and analysis of requirements
- Design (of schemas and applications)
- Implementation
- Validation and testing
- Maintenance while in production

Of course we focus on design of database schemas

Pre-requisites for database design

- Requirement analysis is the responsibility of a specialist (Requirement Analyst) and is done by interviewing all the stakeholders
  - The more stakeholders are active in defining the requirements, the faster the convergence to a satisfying design
- The output of this stage is a **summary text** that describes the requirements (that is typically quite **ambiguous**)
- The text is often completed by:
  - Glossaries and collections of informal definitions
  - Abstract descriptions of the applications that shall use the database and their interactions (dataflow diagrams, use case diagrams, …)

Design Phases

- **1. Conceptual design**
  - Translates the summary of requirements into a formal model: a conceptual schema that shall be independent of any DBMS choice (for us: an E-R model)
- **2. Logical design**
  - Translates the conceptual schema into a logical schema, expressed in one of the many logical models (Hierarchical, Relational, Object-Oriented, XML, …)
- **3. Physical design**
  - Completes the logical schema with whatever detail is required to assure efficiency (tuning of physical access structures, indexes, …)
  - This is widely dependent of the specific DBMS of choice

Dependencies: DATA MODEL and DBMS

	Dependent of the **DATA MODEL**	Dependent of the **DBMS**
Conceptual d.	**No**	**No**
Logical d.	***Yes***	**No**
Physical d.	***Yes***	***Yes***

The database Design Process

Methodologies and Strategies

Design strategies

- There are basically two (opposite) strategies:
  - **Top-Down**
    - Progressively refining very abstract/incomplete schema that already describes all the relevant aspects
  - **Bottom-Up**
    - Progressively integrating small but complete schemas that describe partial, «local» aspects of the scenario of interest
- In particular, we will discuss:
  - An «**oil stain**» approach
  - A «**mixed**» approach

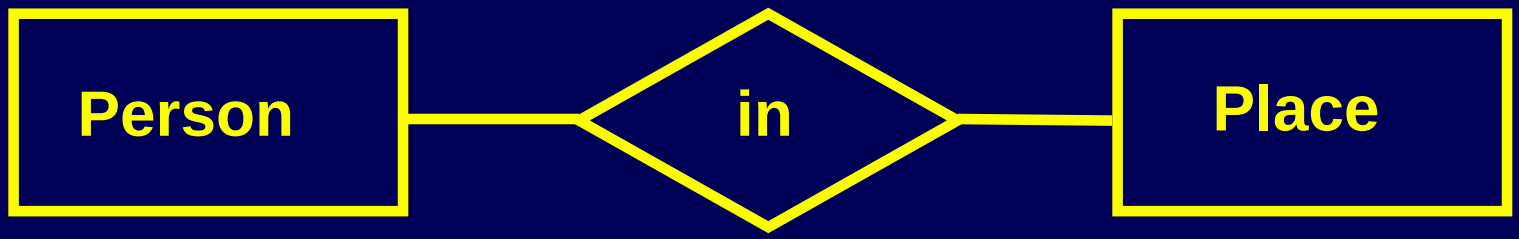
Comparison of the two strategies

- Top-down:
  - ***Pro***: a more ***coherent***, clean, and rational design process
  - ***Con***: generally ***harder***, especially for large projects, as the designer(s) must have a comprehensive perspective on the entire domain of interest
- Bottom-up:
  - ***Pro***: ***easier***, as each expert can focus on their own «niche»
  - ***Con***: separate designer may, in their isolated autonomy, take different, incompatible decisions in solving sub-problems, that will generate ***conflicts*** in the integration process (different, incompatible models of the same concepts)

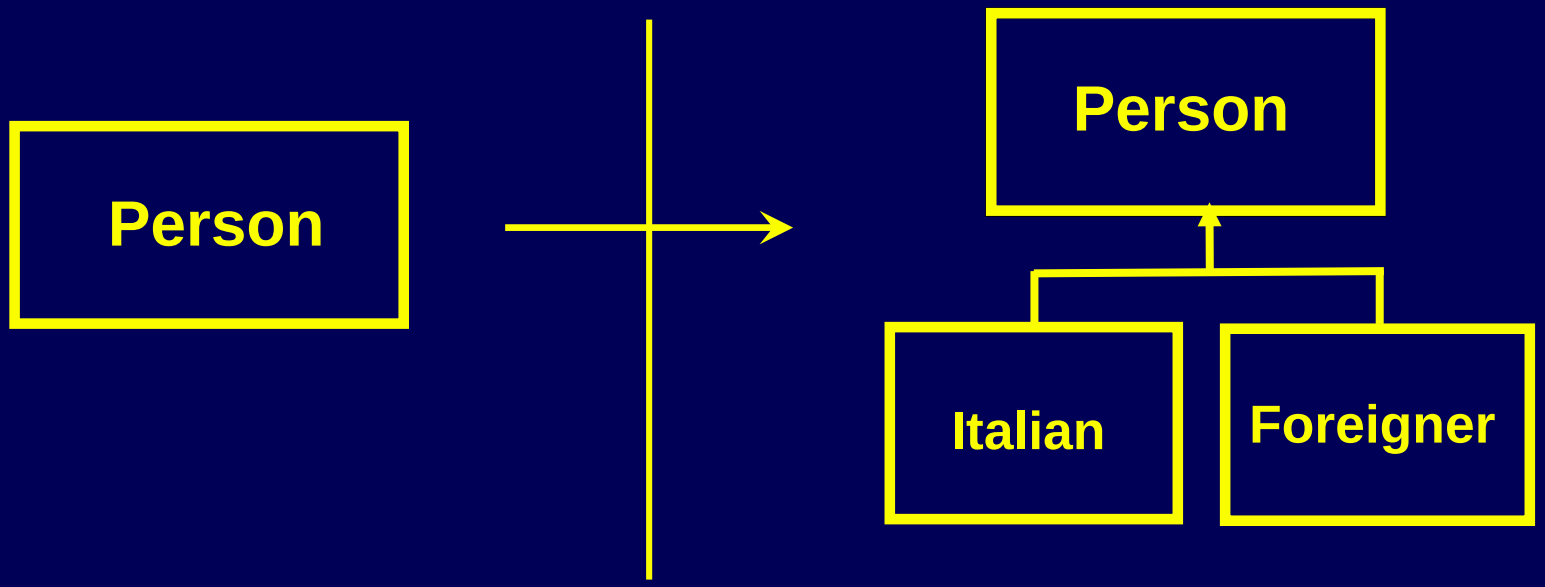
“Pure” top-down strategy

- The reqirements are distilled into an **initial schema**
- Several subsequent refinements lead, through many intermediate schemas, to a **final schema**
- These refinements apply **elementary transformations** (primitives) that are applied to single concepts in order to describe them in a more detailed way

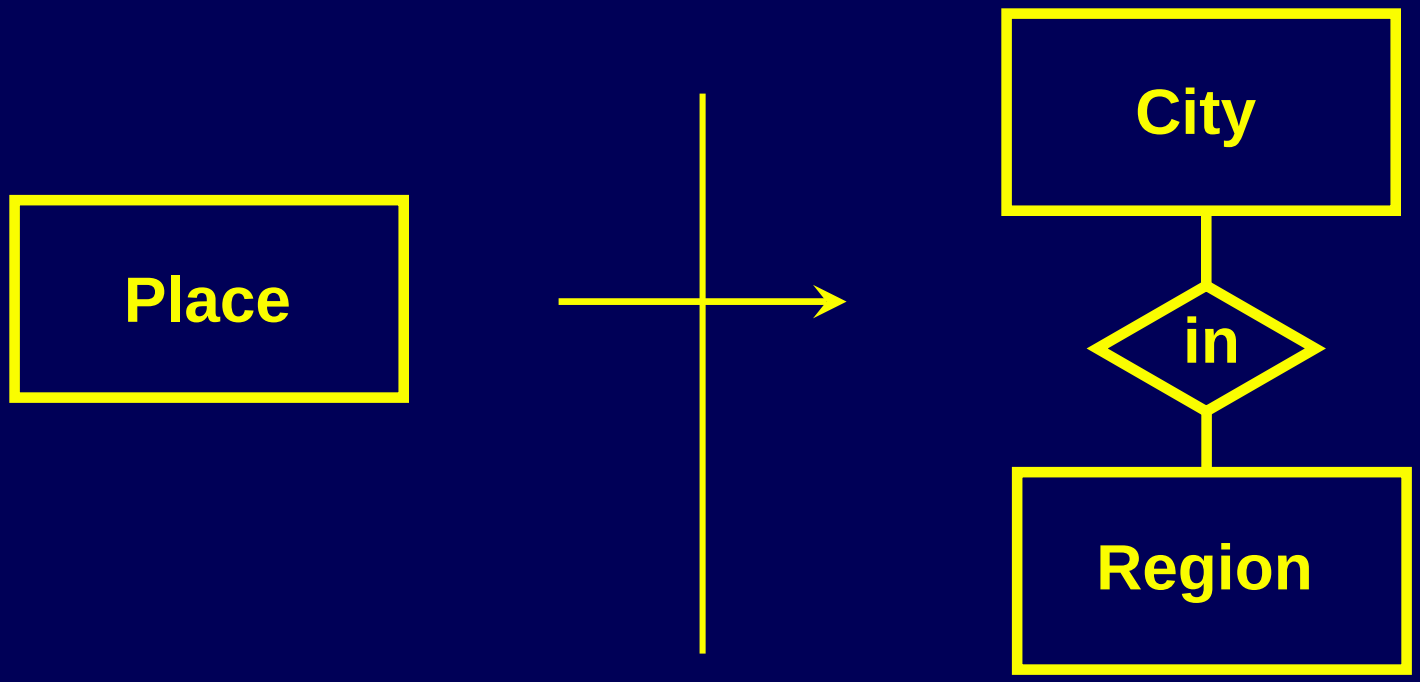
Step 1 – Initial abstract schema



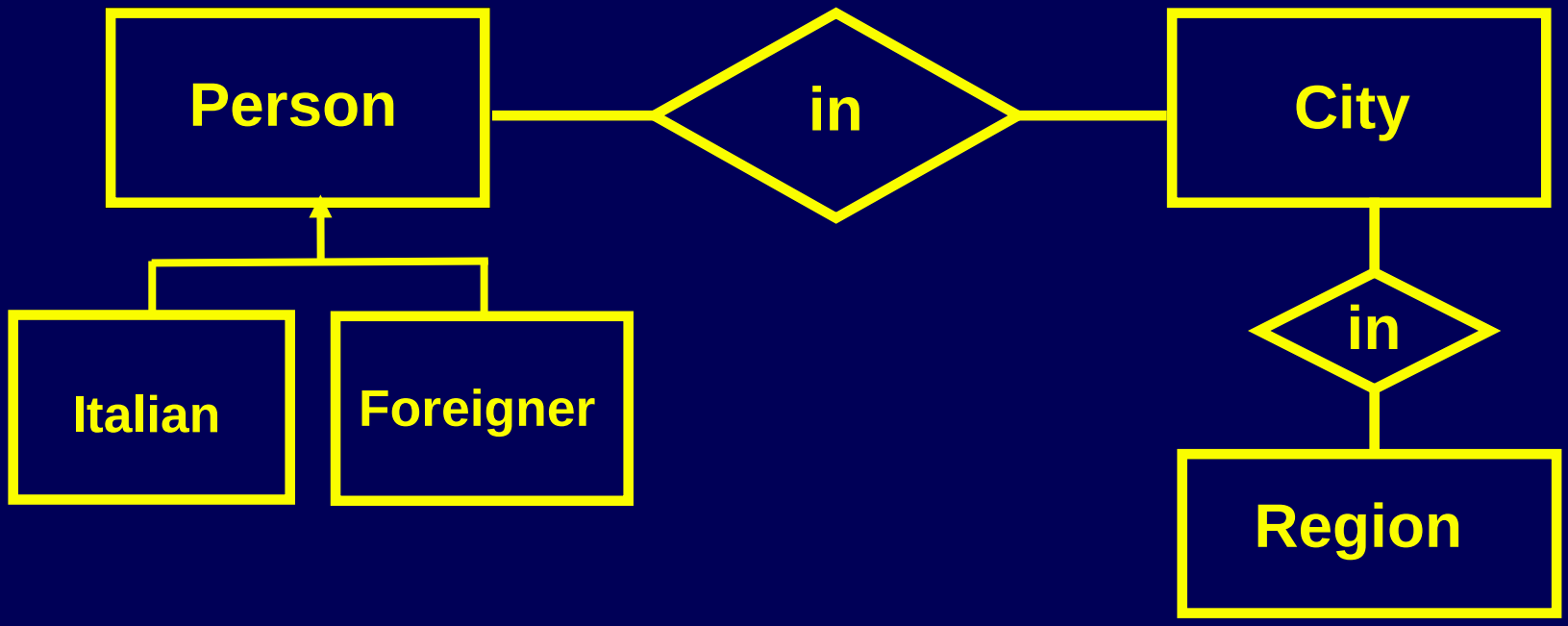
Elementary top-down refinement



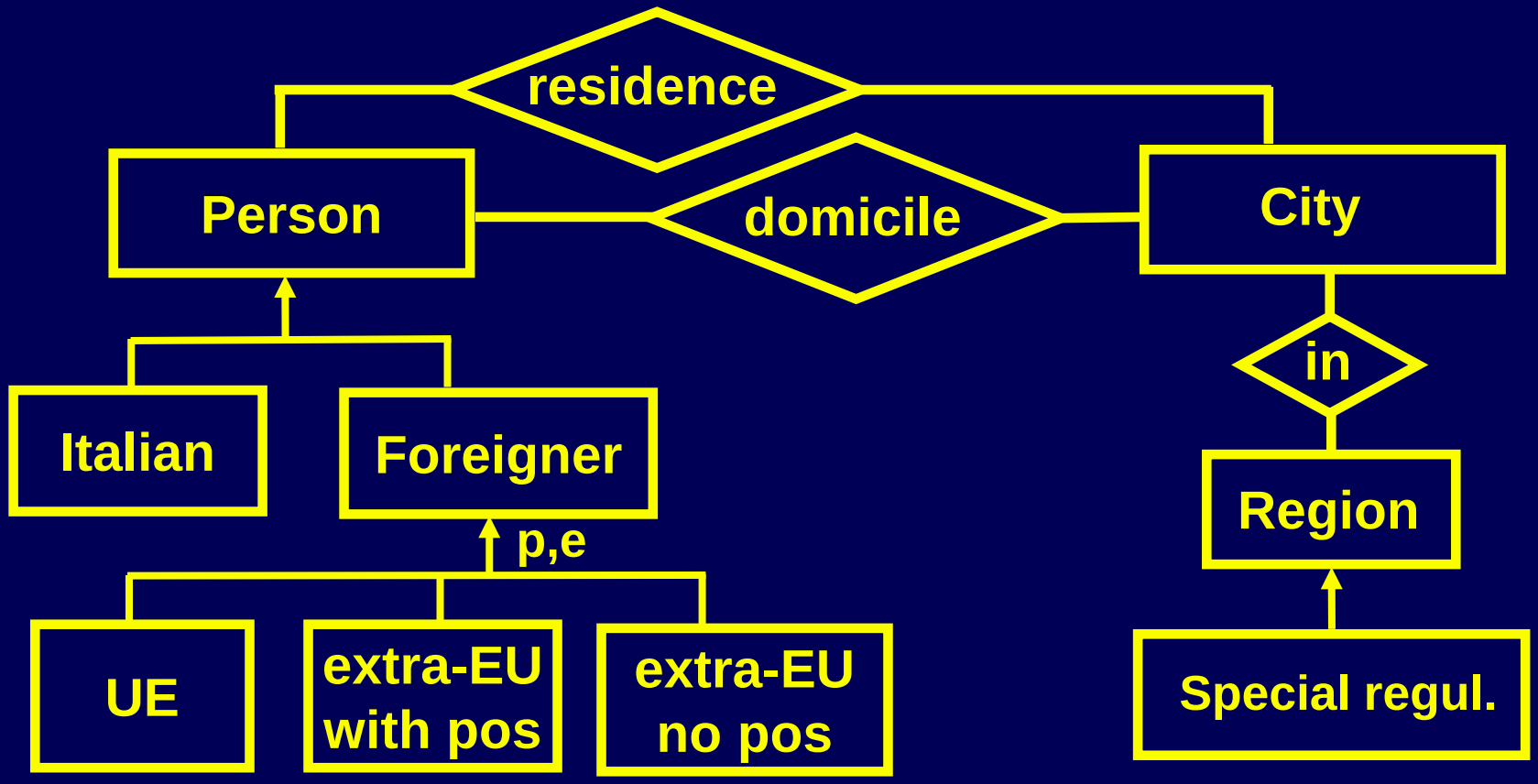
Elementary top-down refinement



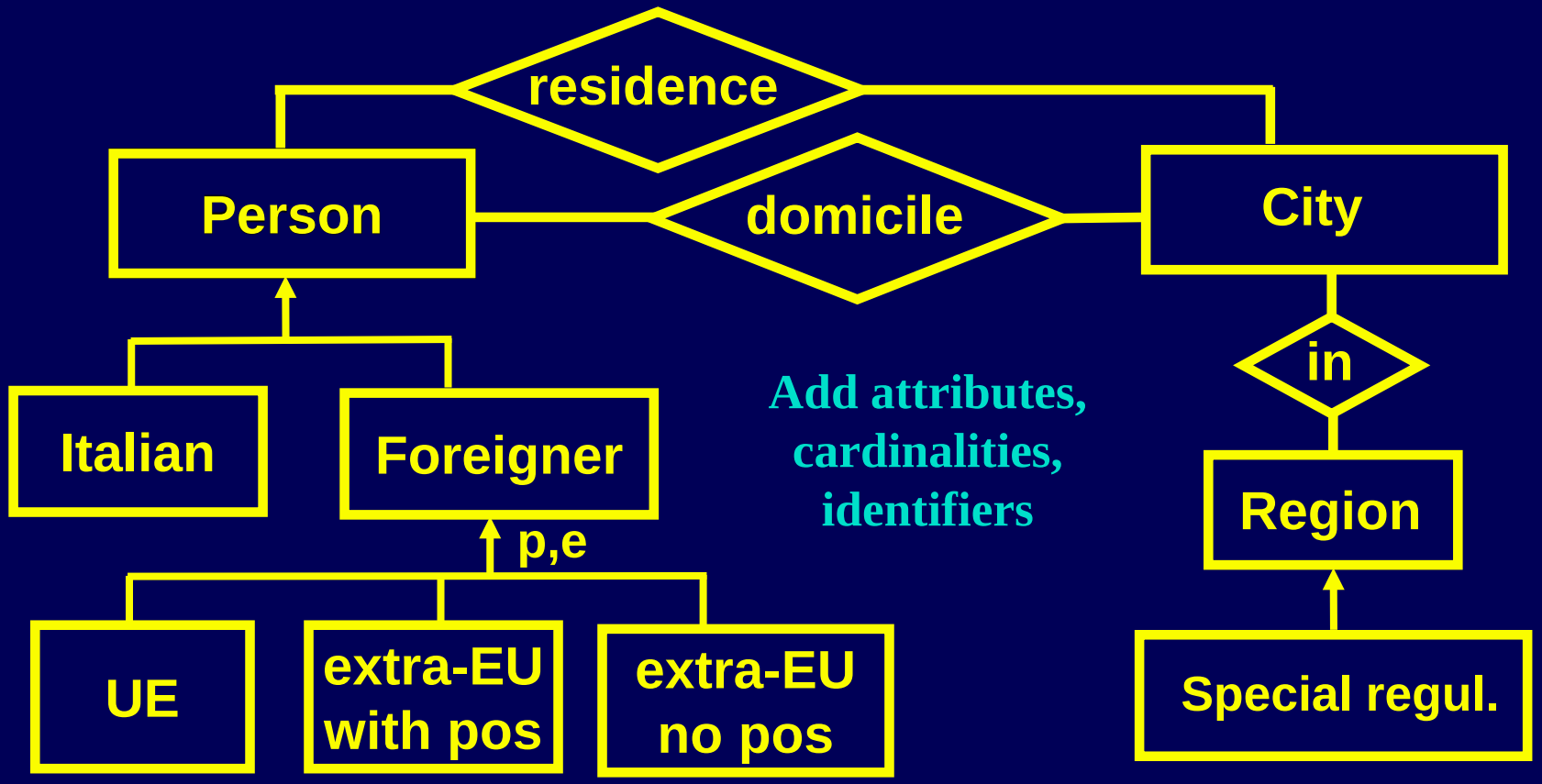
Step 2



Step 3 – further details



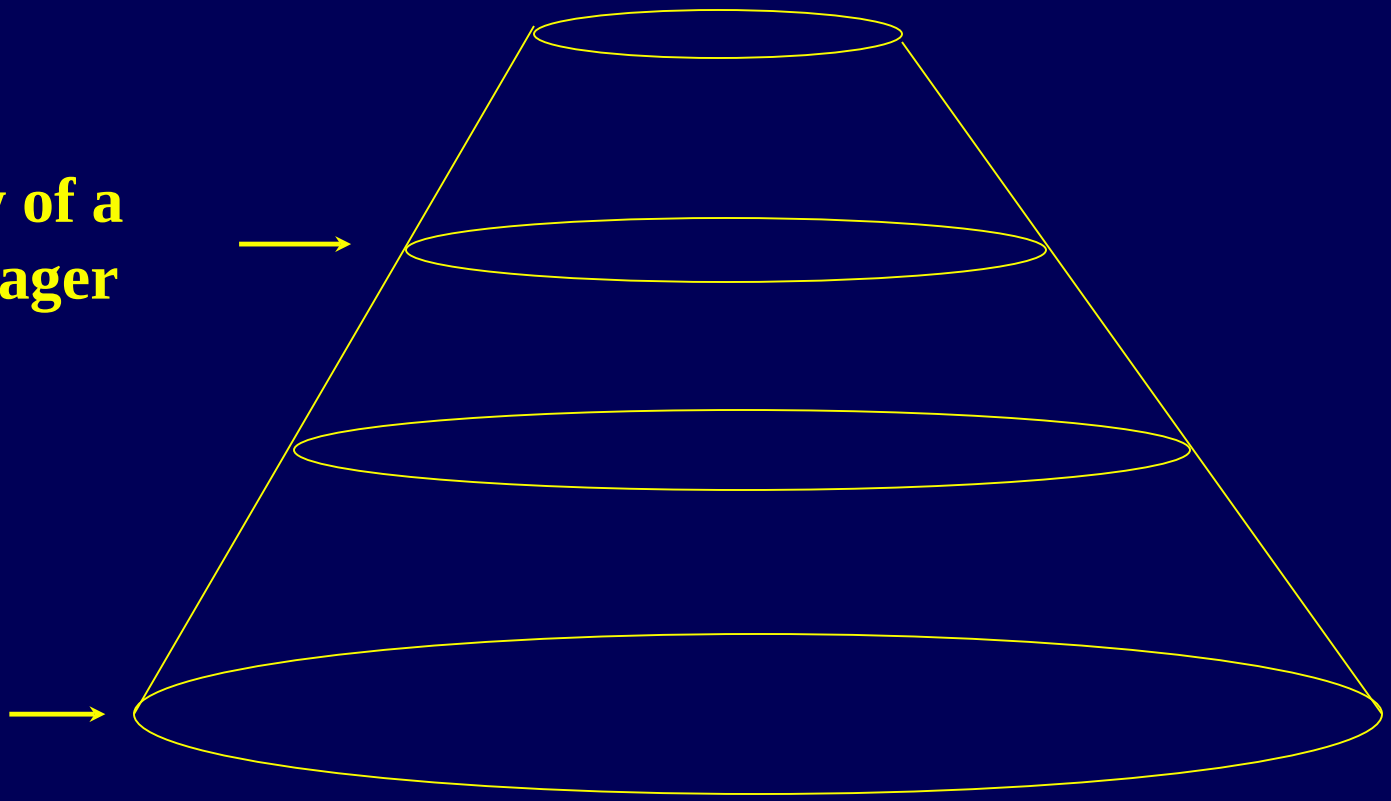
Step 4: decorating with attributes and cardinalities



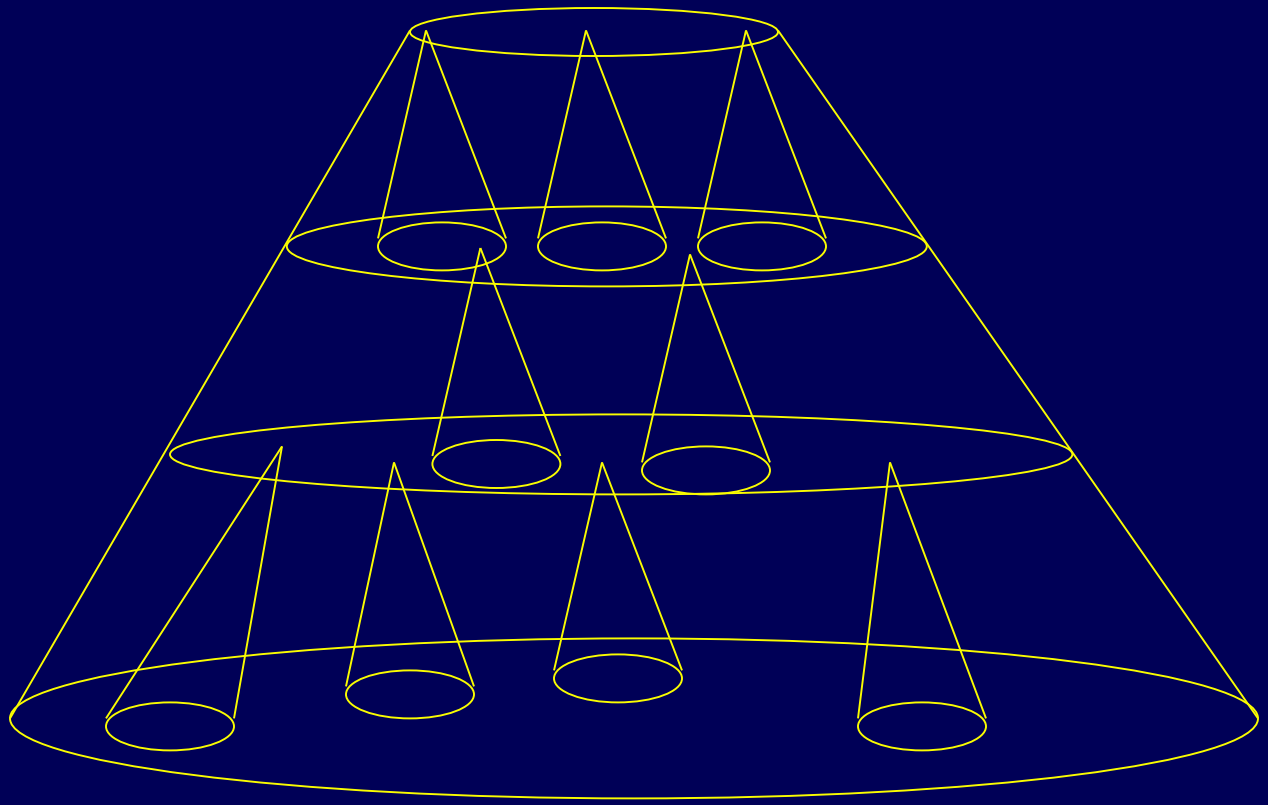
Project “planes”

View of a Manager

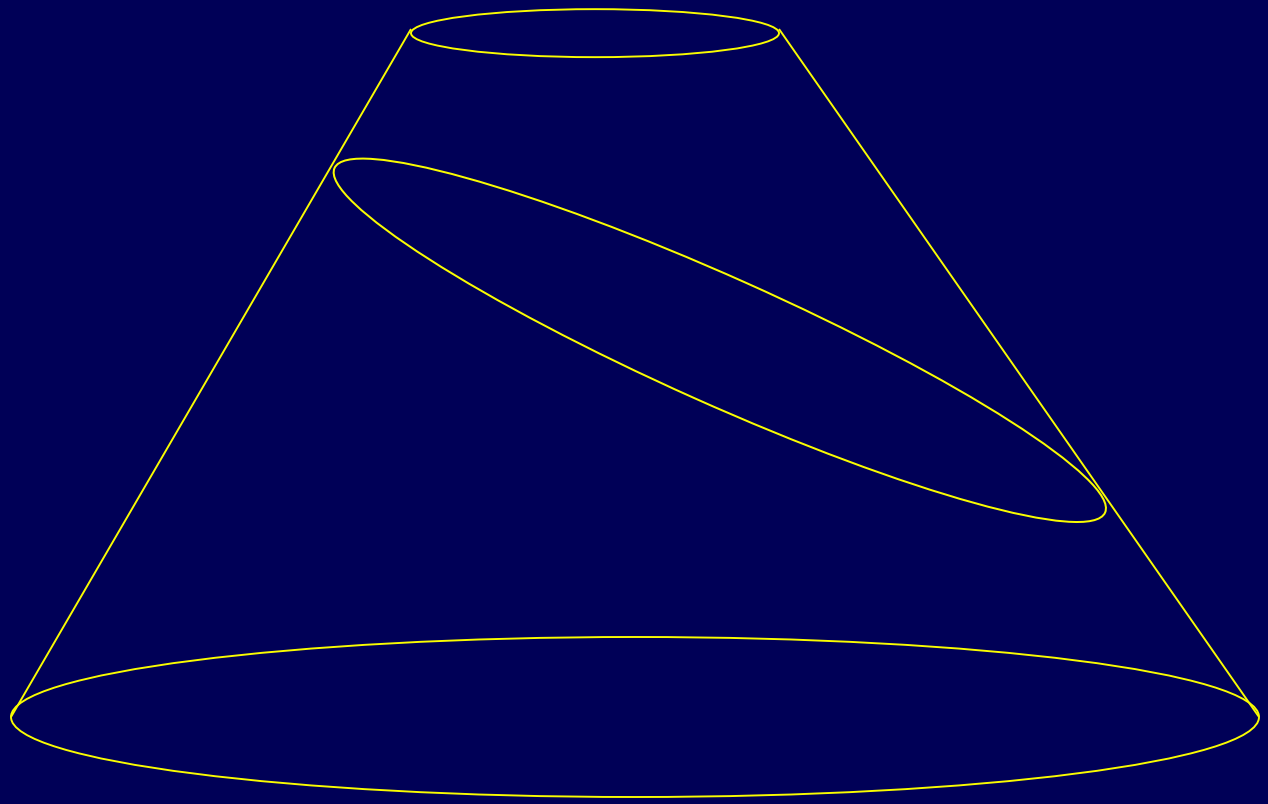
View of IT staff



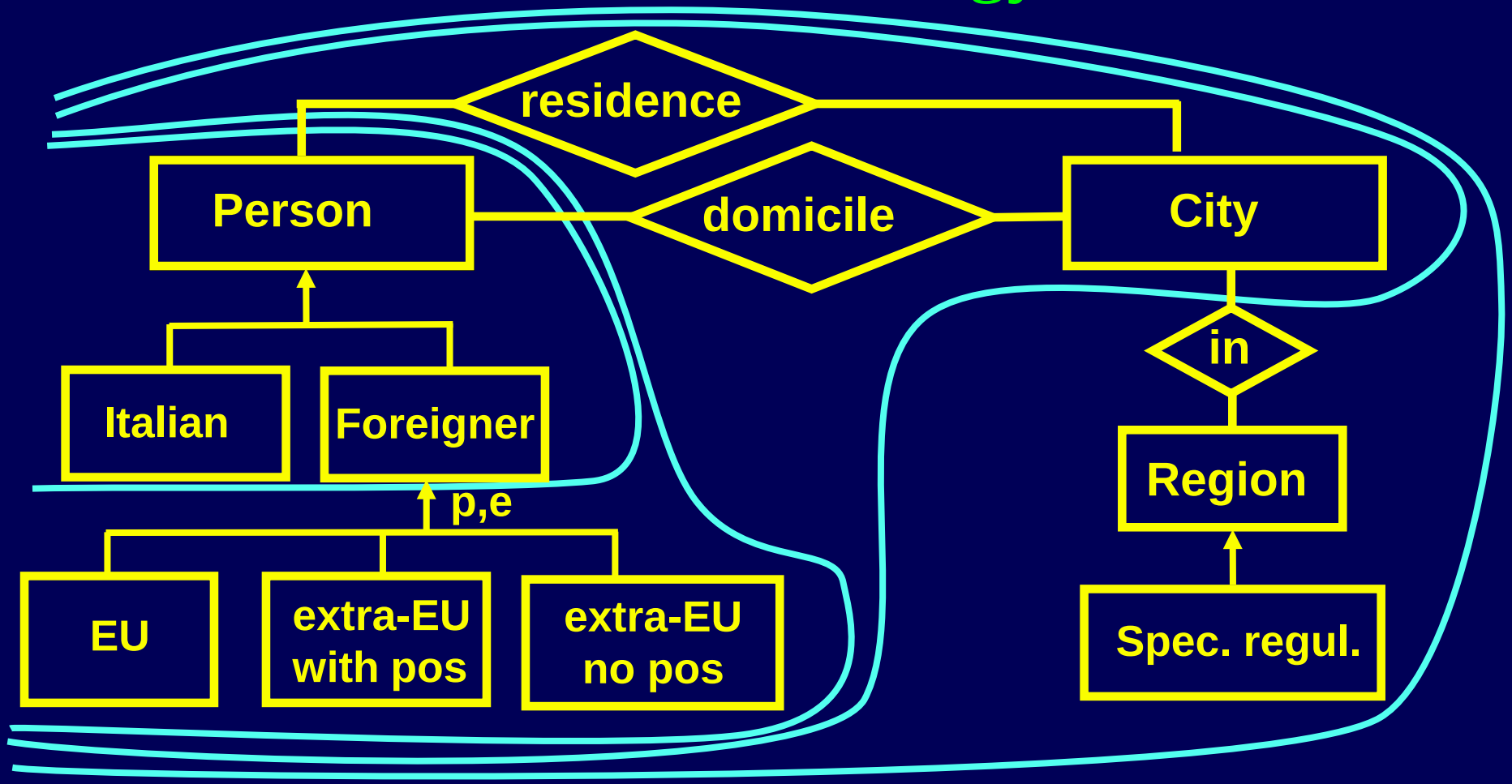
Balanced design



Unbalanced design



“Oil stain” strategy



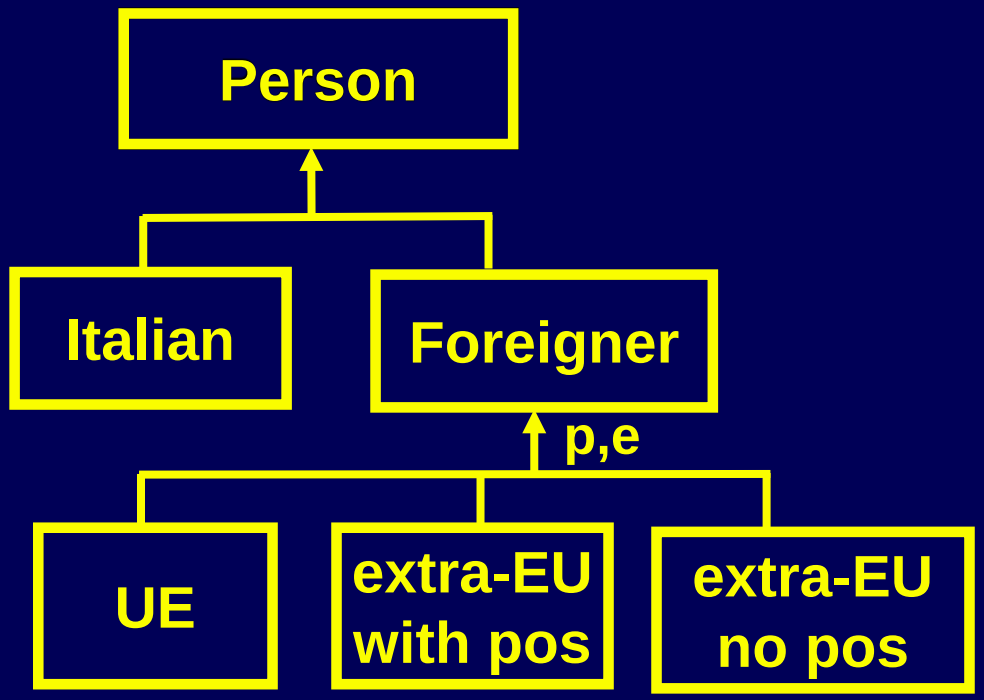
Evaluation of the “oil stain” strategy

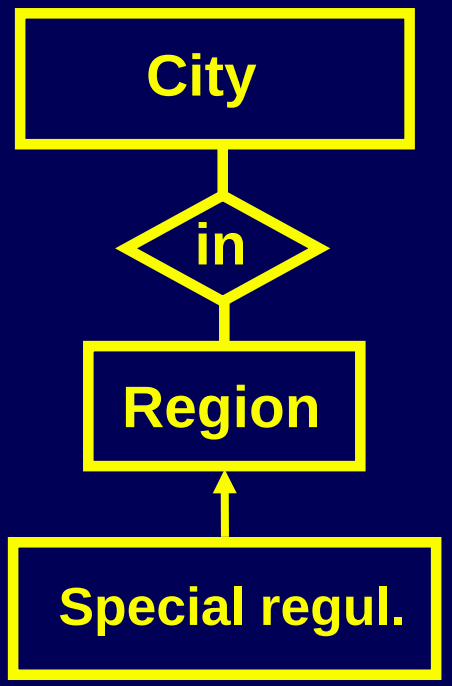
- Specifications are added incrementally, translating the requirements at the finest granularity level, with full details, tackling one sub-problem at a time
- Well suited to ***incrementally*** transform a **textual** description into a **diagram**
- It is essentially bottom-up (each new addition is conceived and integrated into the existing schema), but the designer processes the specifications in a «stratified» way
  - Incremental additions to one schema make conflicts less likely

Mixed strategy

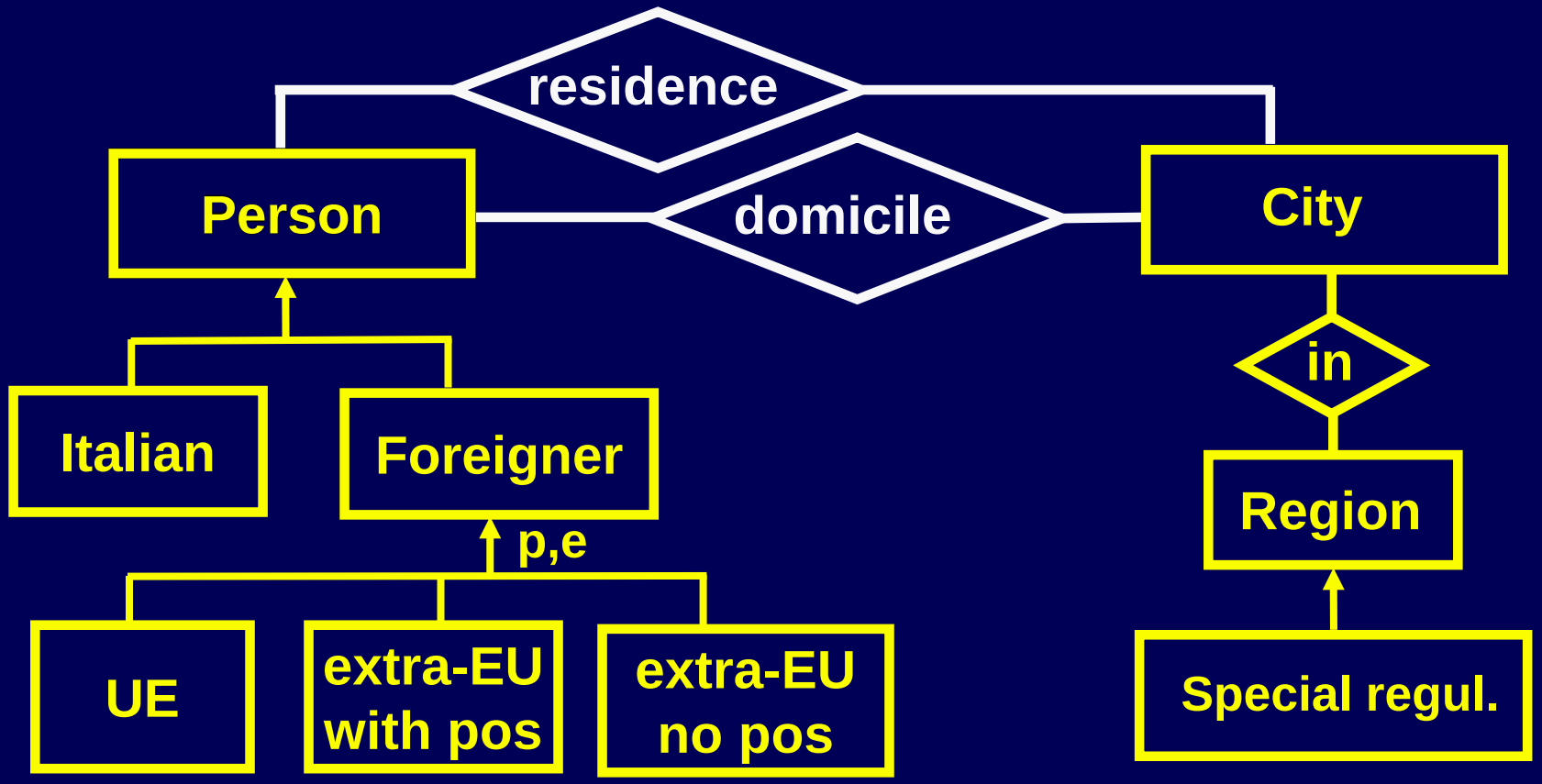
- Well suited for large projects
- The schema design specifications are divided into parts (administration functions, marketing, production, …)
- Each part is designed with a top-down approach
- Integration of the parts is then performed bottom-up

Development of the two sub-schemas





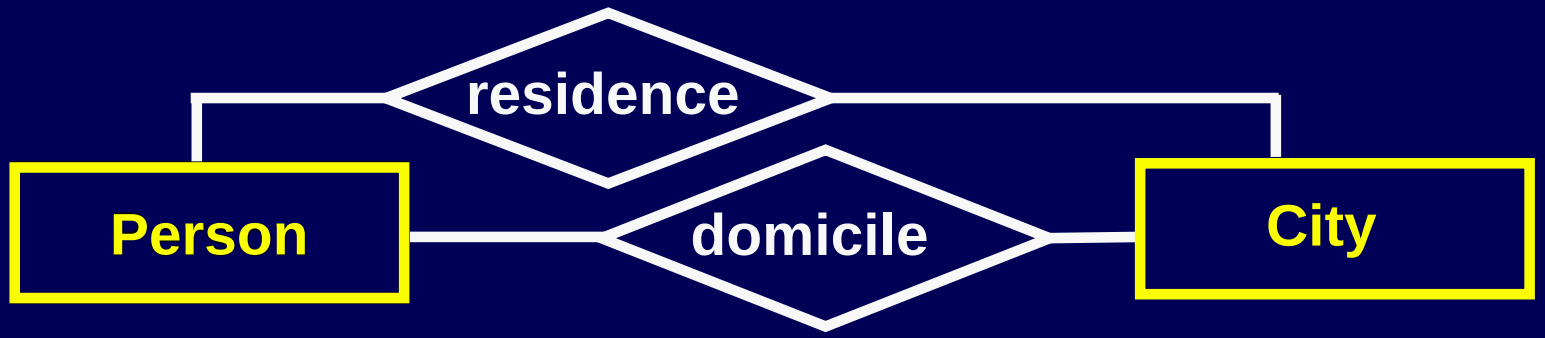
Integration



Evaluation of the mixed strategy

- **Pros:**
  - **The perspective of the various experts on their specific assignment is precise and knowledgeable**
- **Cons:**
  - **Semantic conflicts make integration potentially hard**
- **A possible solution:**
  - **Start (in top-down mode) with the main concepts modeled a small, abstract schema (*skeleton schema*), that is agreed upon, and implement the parts coherently with the initial decisions**

Skeleton Schema



All in all…

- An expert designer continuously switches between top-down and bottom-up, without even realizing it
- In order to solve **our (small-sized!!) exercises**, the oil stain approach is normally the most effective
- In any case, it is possible (and normally convenient) to DOCUMENT ***ex post*** a completed and agreed-upon design process in a top-down way

The database Design Process

Quality of a Conceptual Model

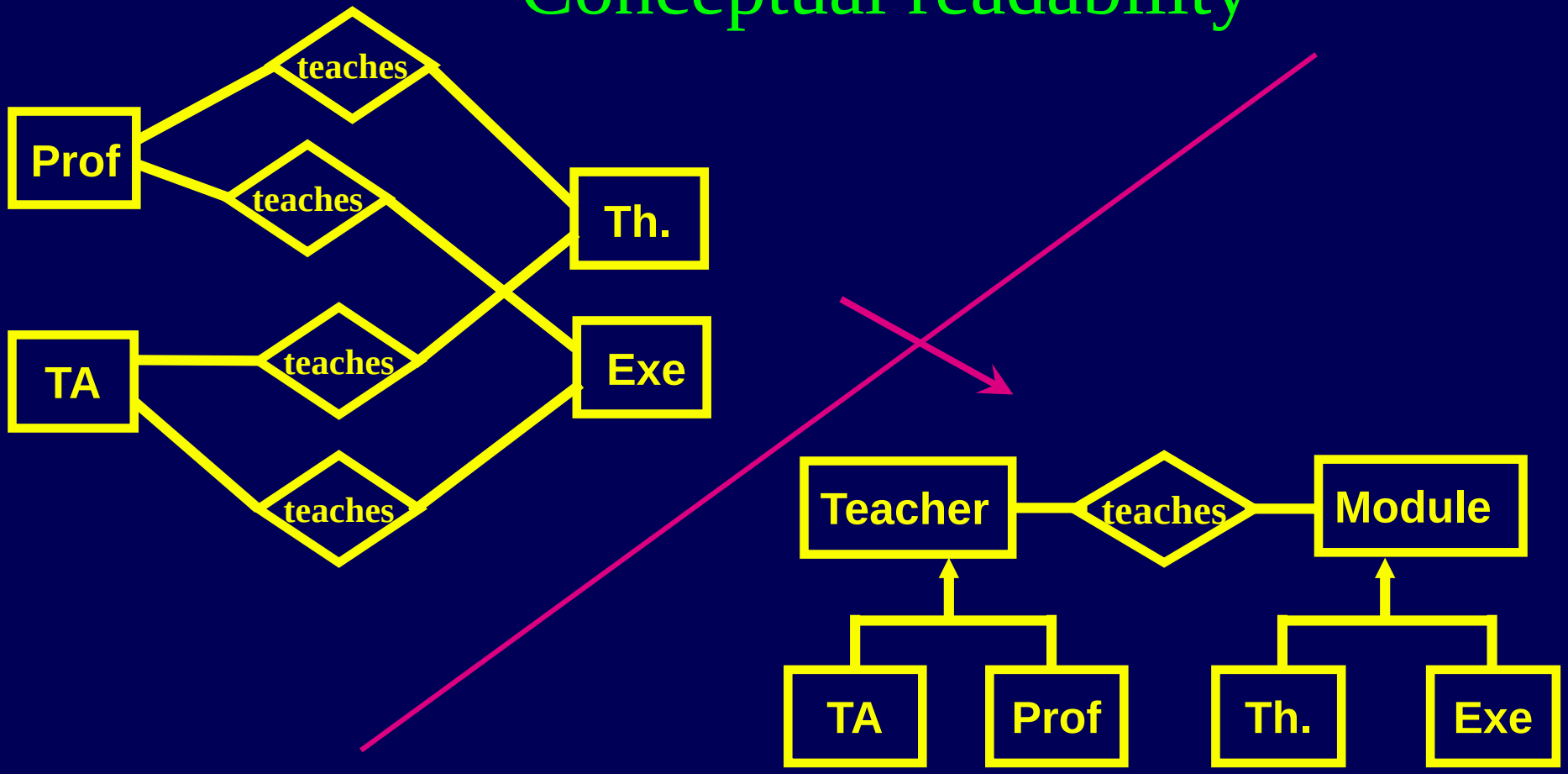
Quality of a conceptual schema

- Completeness
- Correctness
- Readability
- Minimality
- Being self-explanatory

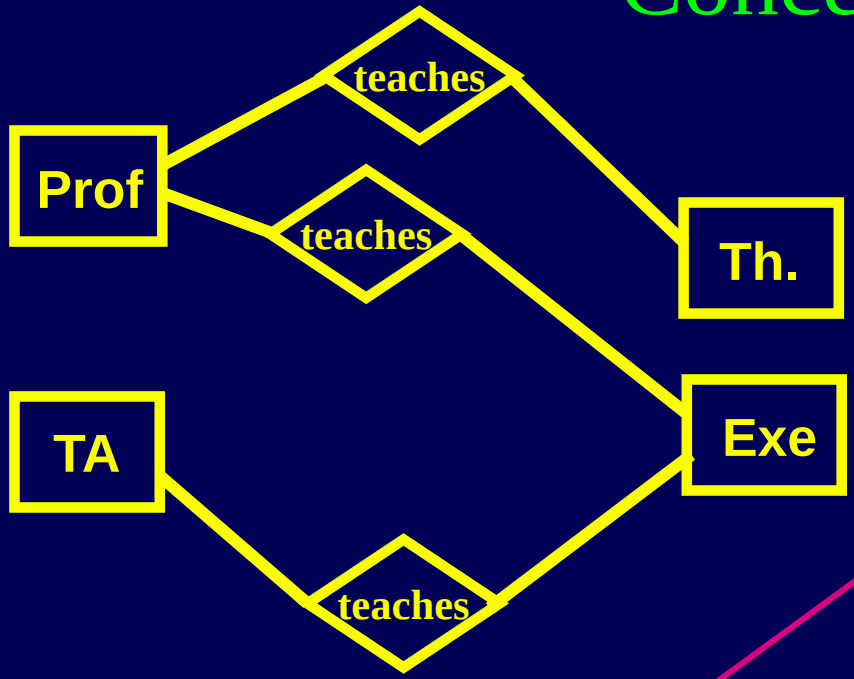
Completeness and correctness

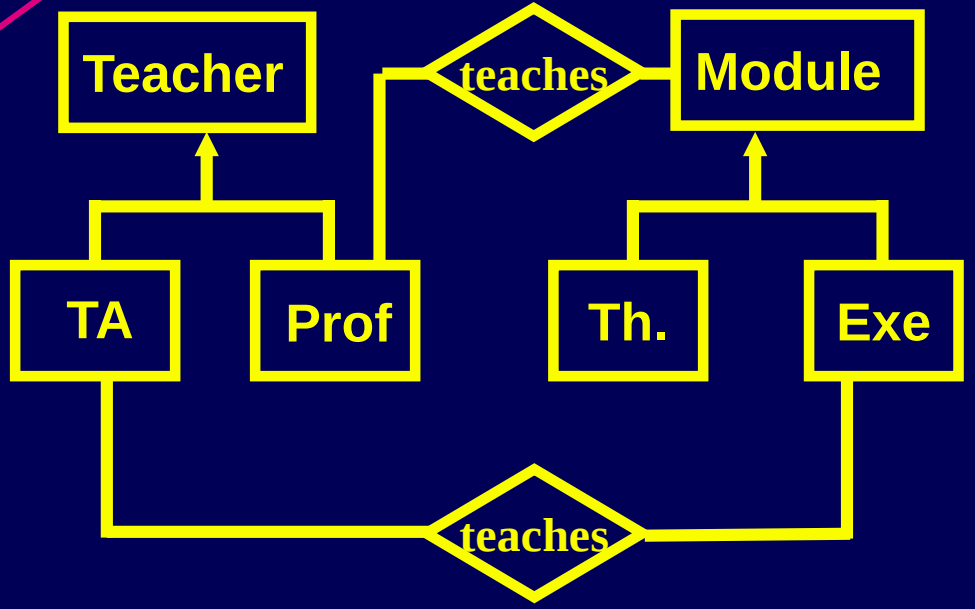
- Requirements shall be represented **exhaustively** and **correctly**
- These properties are quite straightforward!
  - For **completeness**: data should be modeled in such a way that all applications are allowed to run
  - For **correctness**: it should be possible to populate the database with partial, incomplete data during the initial phase of its evolution

Conceptual readability

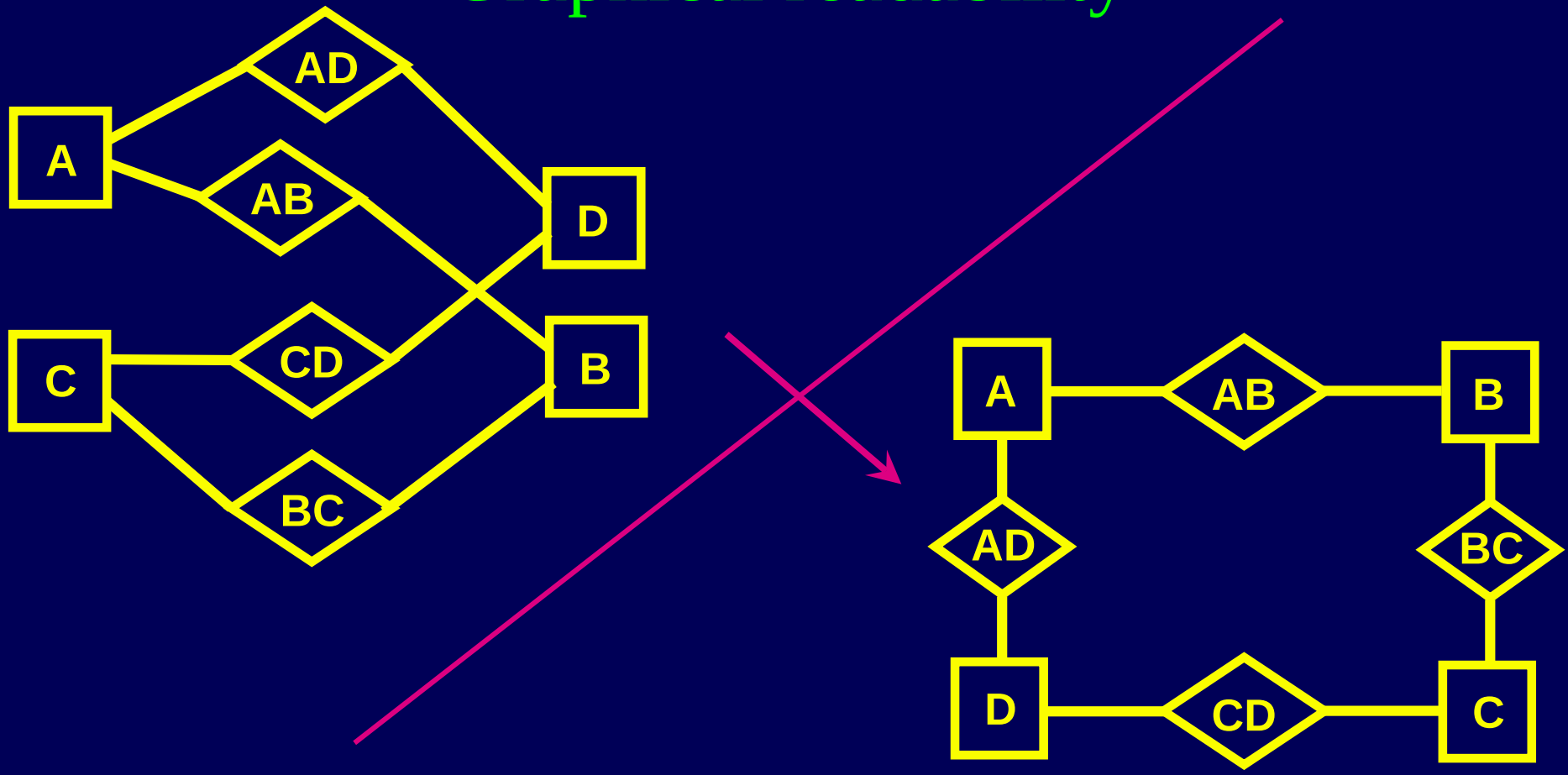


Conceptual readability

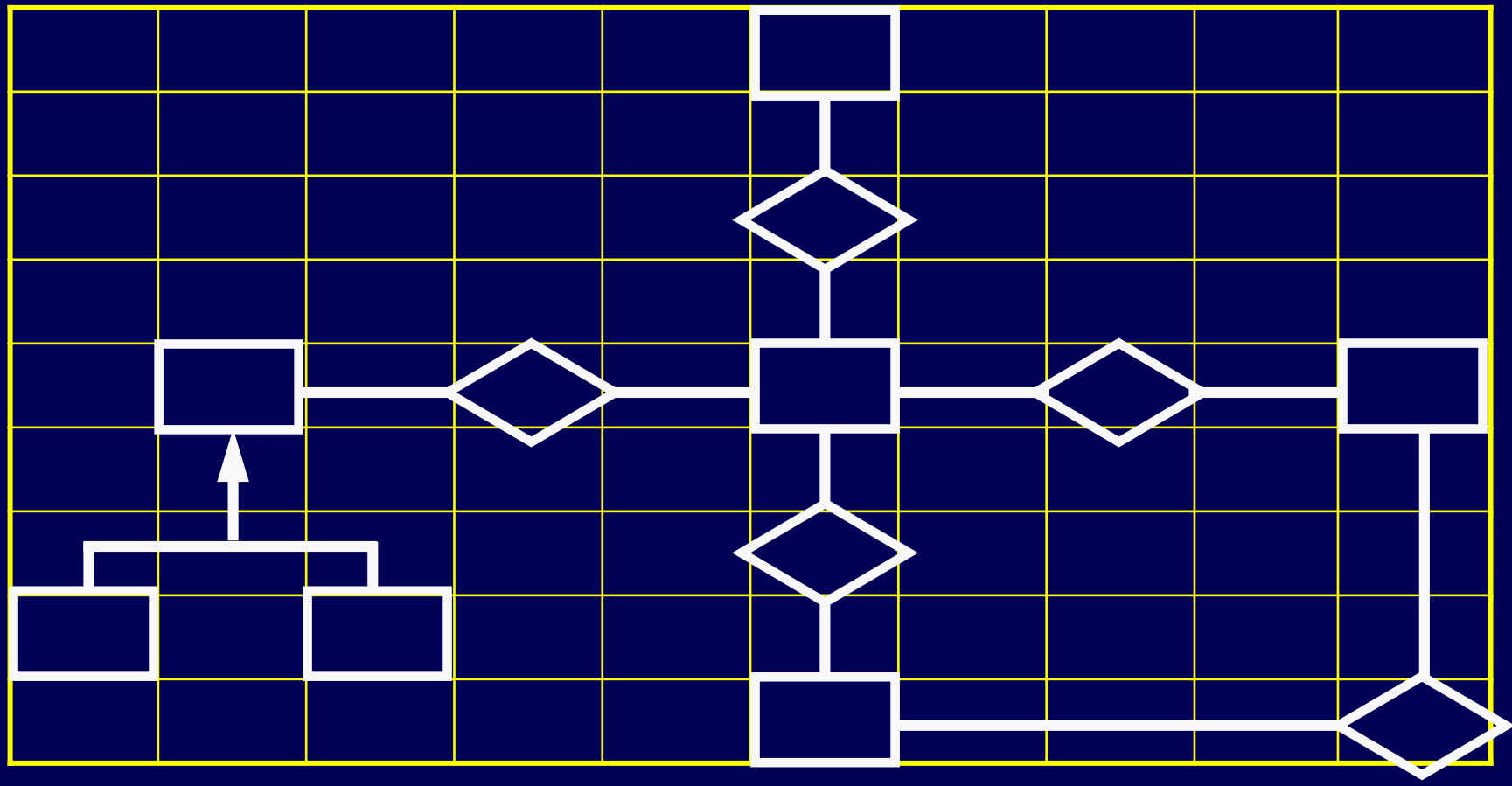




Graphical readability



Simple and smart: drawing in a grid!



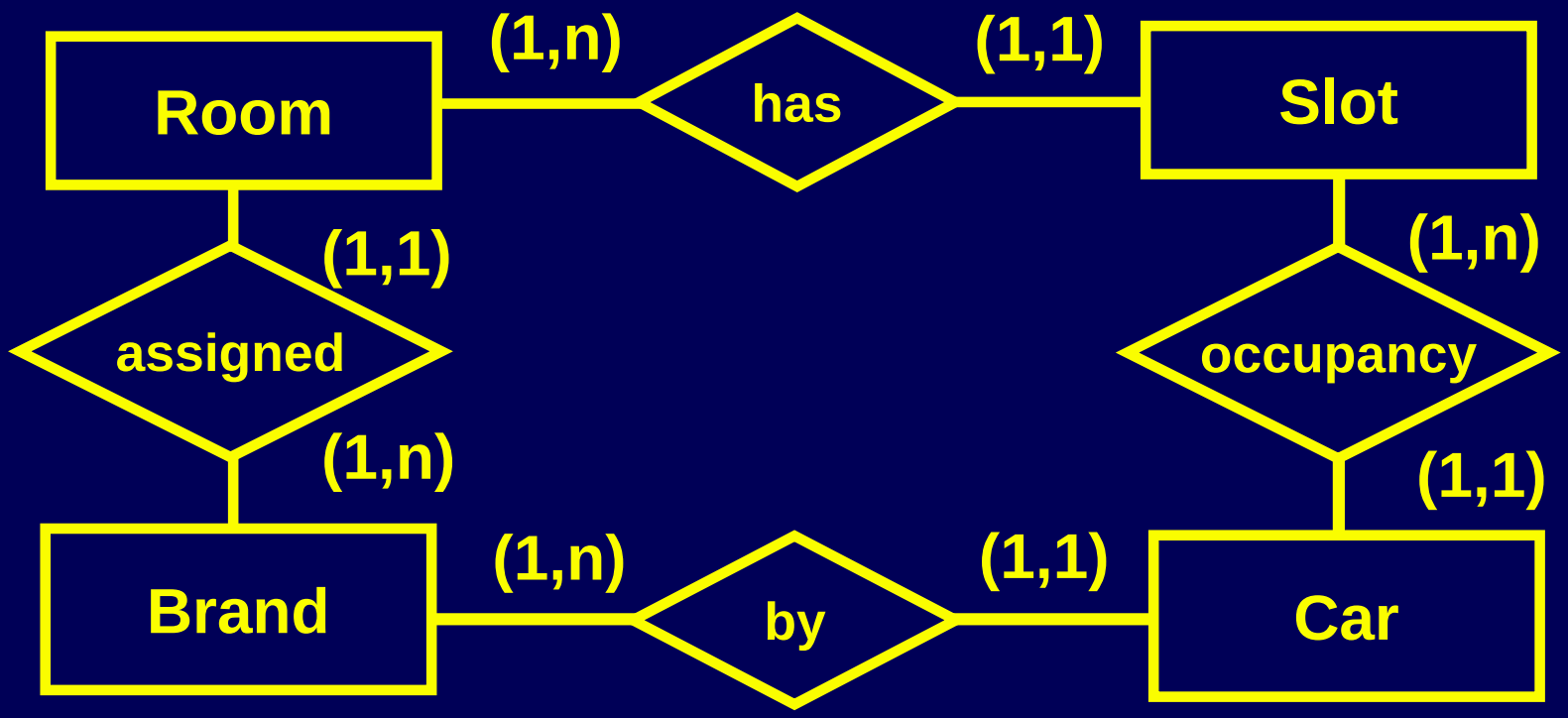
Redundancy within schemas

- Should be at least identified and documented
  - Not necessarily eliminated
- Example: the AMOUNT in a BANK-ACCOUNT
  - It is possible to derive the amount from the initial amount and the algebraic sum of the amounts of all transactions occurred on the account ever since (or since the latest balance)
  - However, it is logically (and computationally) convenient to have this piece of information replicated, already computed, easily accessible, and… up to date!

An example of potential redundancy

- **An expo company has show-Rooms for cars**
- **Rooms are *normally* assigned and fully dedicated to Brands (automobile manufacturers)**
- **Rooms have Slots, each occupied by one or more Cars**
- **Cars are produced by brands**

Redundancy within schemas

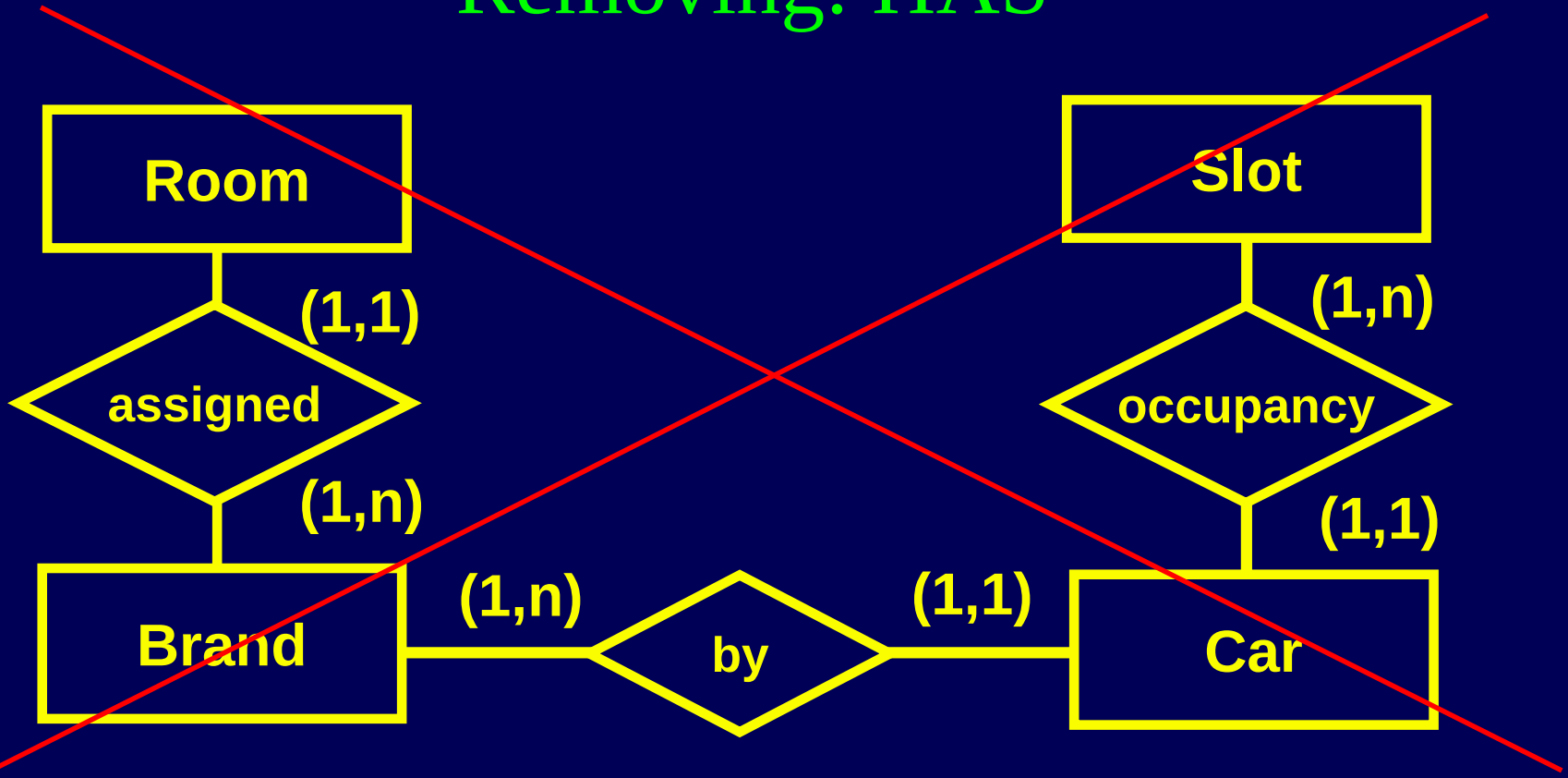


There is a **cycle**: there may be **redundancy**

Redundancy within schemas

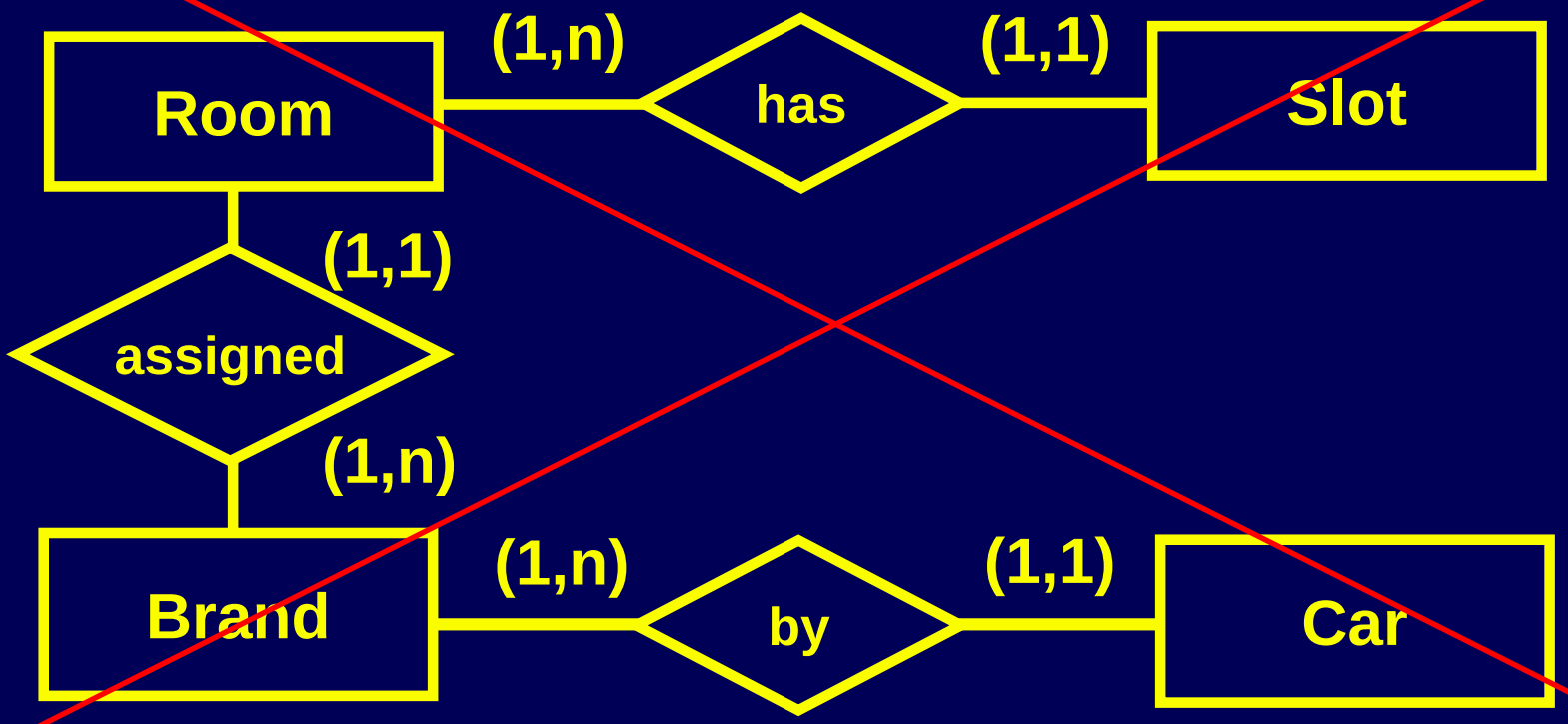
- **The cycle is indeed redundant if cars are placed in slots with the constraint that a room is always exclusively assigned to the cars of a single brand**
- This can be shown by braking the cycle in all possible ways (by eliminating each association in turn) and checking whether all requirements are still satisfied

Removing: HAS



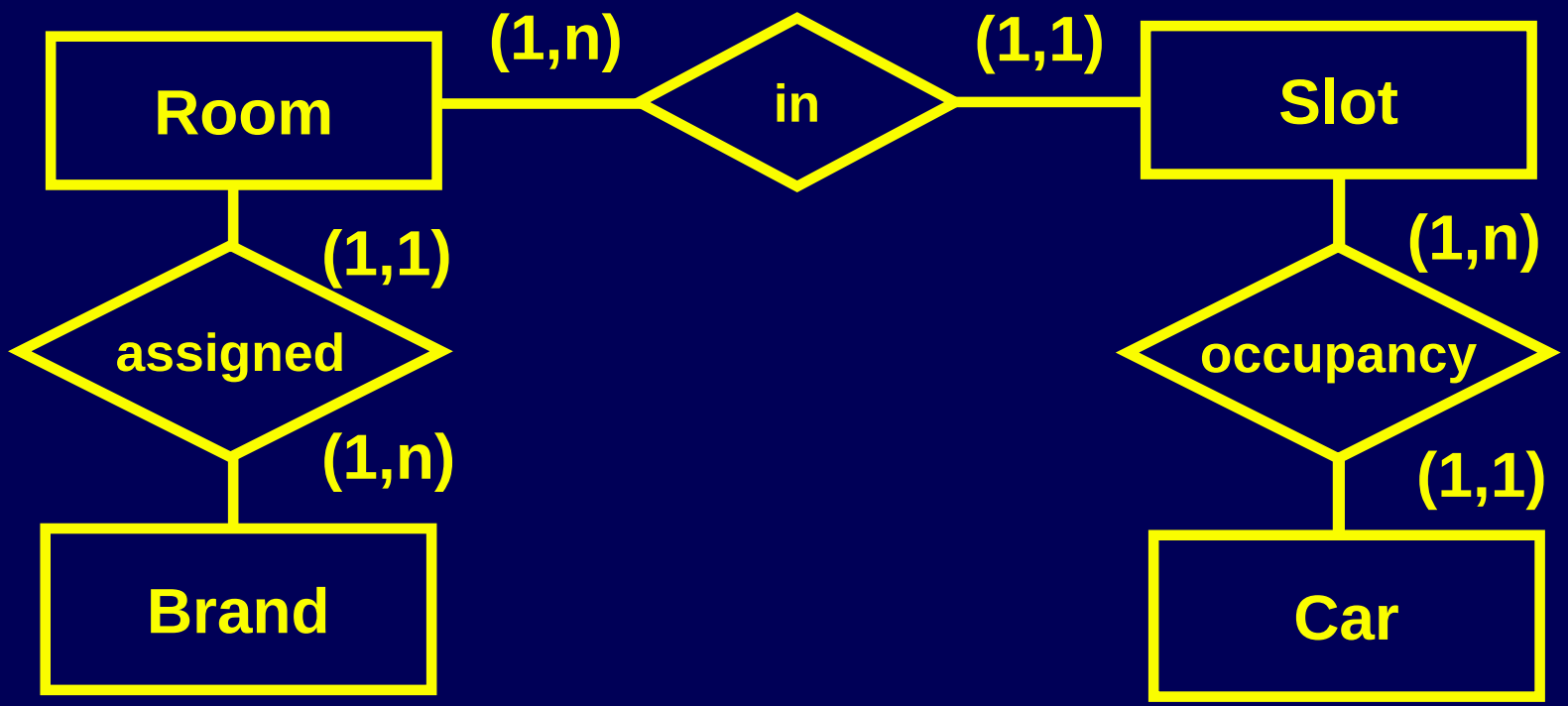
Given a slot, it is not possible to identify the room of that slot

Removing: OCCUPANCY



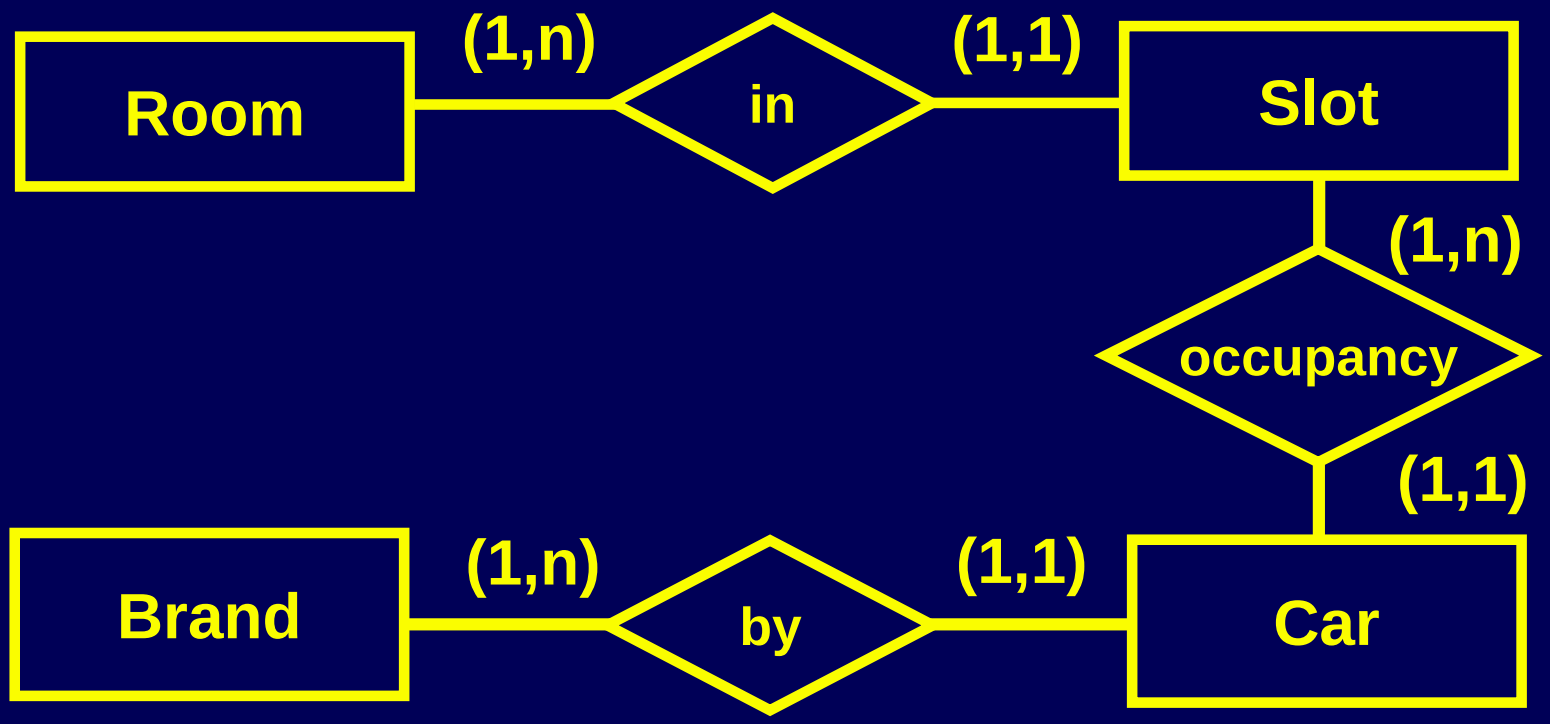
Given a car, it is not possible to identify the slot of that car

Removing: BY



No information is lost!

Removing: ASSIGNED



Once more, no information is lost (not so obvious!)

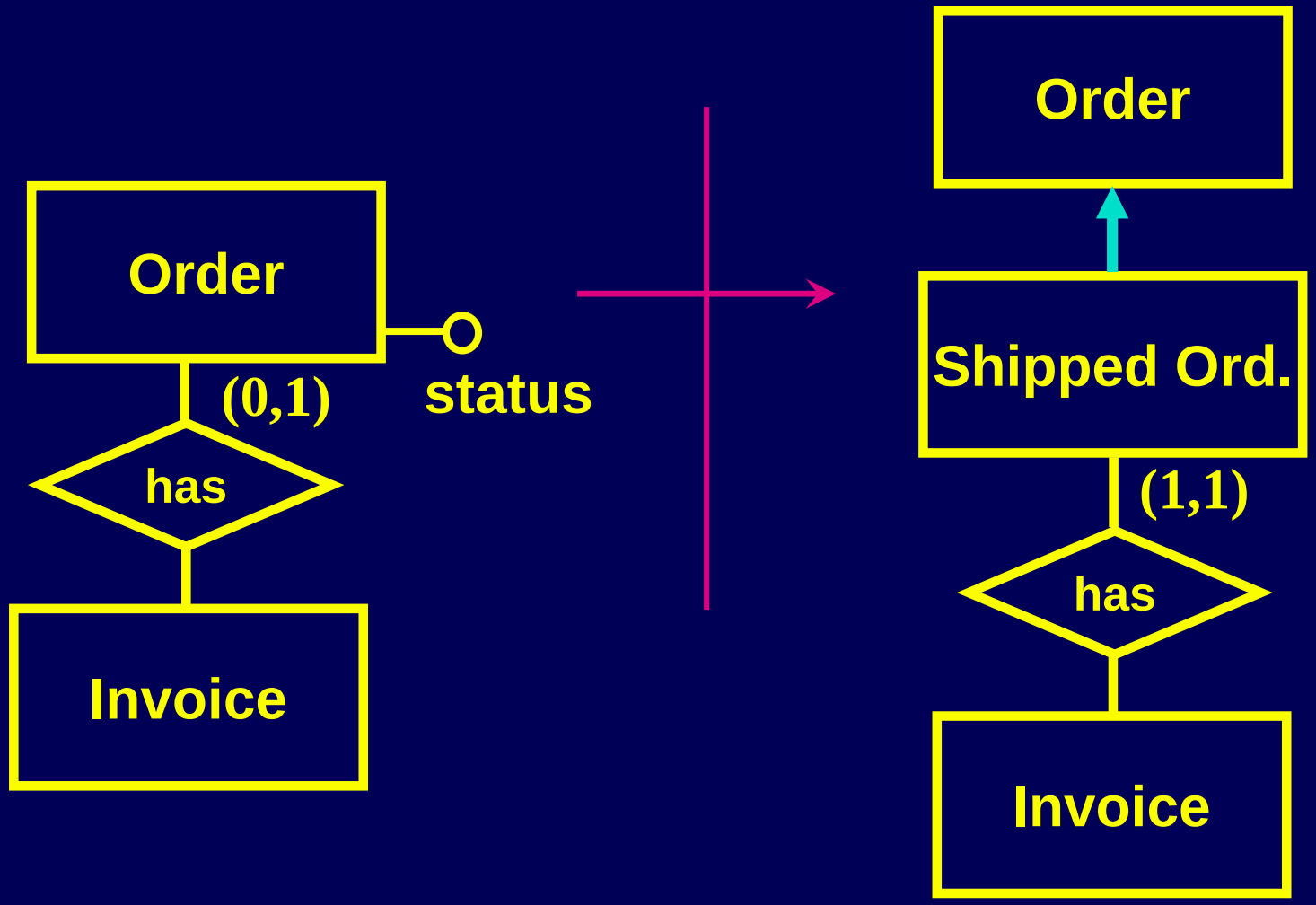
The «exclusive assignment» constraint is not explicit in the schema

Discussion

- It is often unreasonable to eliminate all redundancy
  - It is then necessary to resort to complex computations to reconstruct the implied information (with a complex «chain of joins»)
- Anyhow… it is extremely important to understand precisely the meaning of associations!

Being minimal and self-explanatory

- Schemas should explicitly represent as much knowledge as possible about the domain of interest, with the minimum number of constructs
  - Terseness facilitates quick and clear understanding
- Schemas should be easily readable and understandable without long explanatory texts
  - Otherwise their main goal, ***quickly and effectively communicate a conceptualization***, is intrinsically not achieved


Order
(0,1)
status
has
Invoice
Order
Shipped Ord.
(1,1)
has
Invoice

So far....

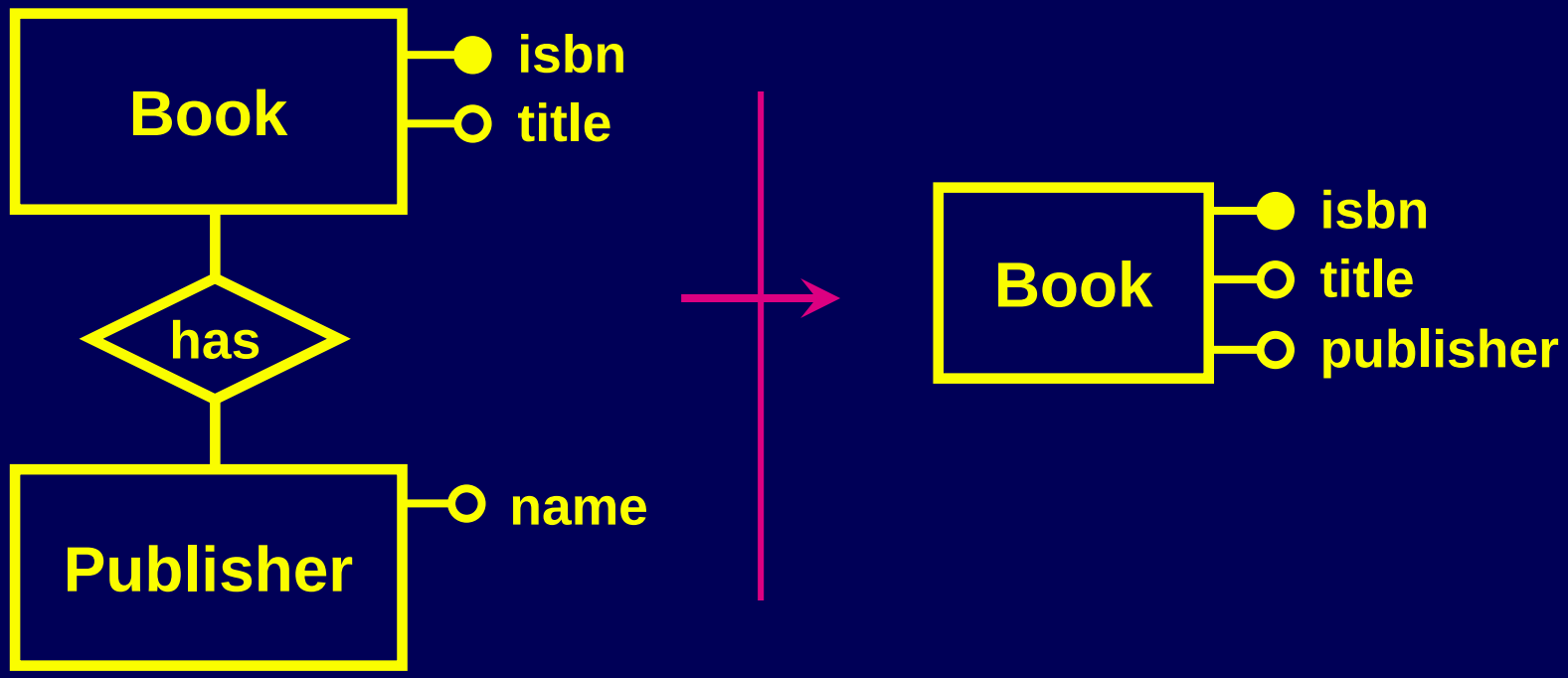
- The design was carried out (or documented) with a top-down strategy
- Quality requirements are reasonably matched

The last step is a final revision (post-processing)

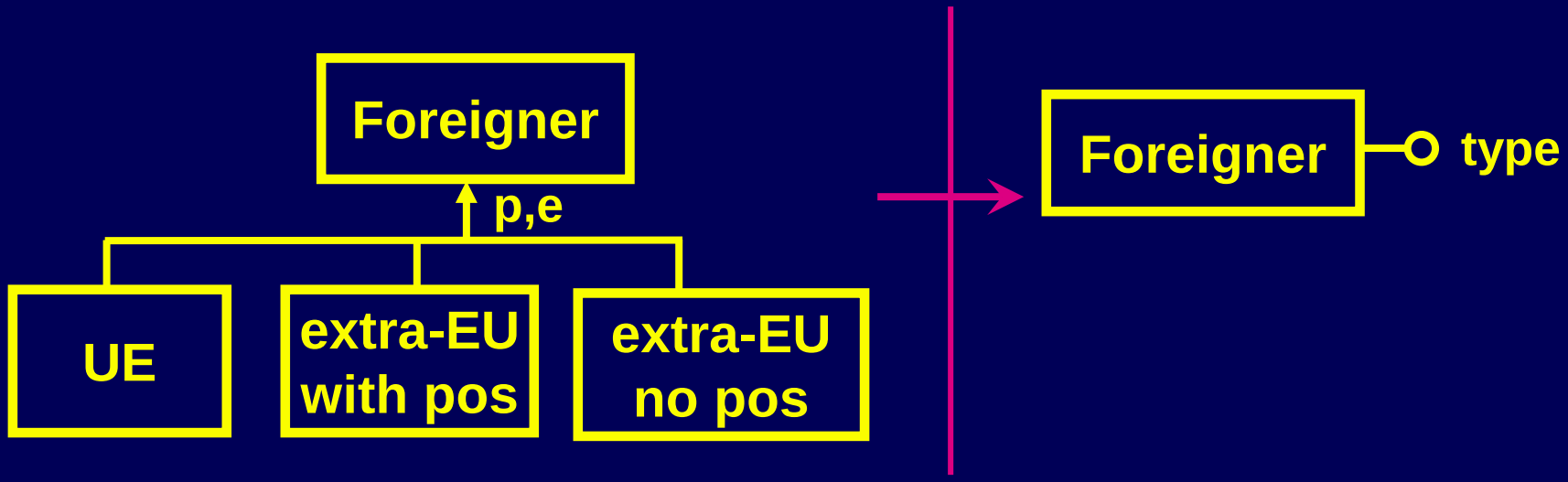
Post-processing

- Final checks:
  - All entities must have at least one identifier
  - All associations shall have their cardinalities
  - All entities must be «meaningful»
    - Having relevant attributes and/or being connected to other entities by relevant associations
  - Generalizations must be useful
    - Allowing inheritance of relevant properties
    - Having meaningful specialized entities

A useless entity



A useless generalization



Logical Database Design

a) **Conceptual Database Design**
   - The E-R Model

b) **Design principles and Methodologies**
   - Life cycle of databases within Information Systems
   - Quality of conceptual models

c) **Logical database Design**
   - Translating from the E-R model into the Relational Model

d) **Normalization**
   - Functional dependencies and normal forms

e) **Physical Database Design**
   - (hints to) Access structures, Indexing, Query plans

Abstraction levels

- An E-R schema descrbes an application domain at a specific (high) abstraction level
- Its main purposes are:
  - Giving a visual, terse description of the domain
  - Fixing the terminology and represening a significant part of the semantics of the involved applications
  - Fostering information exchange between team members about design activities, and support the maintenance of design documentation
- But how does it translate to a logical (relational) schema?

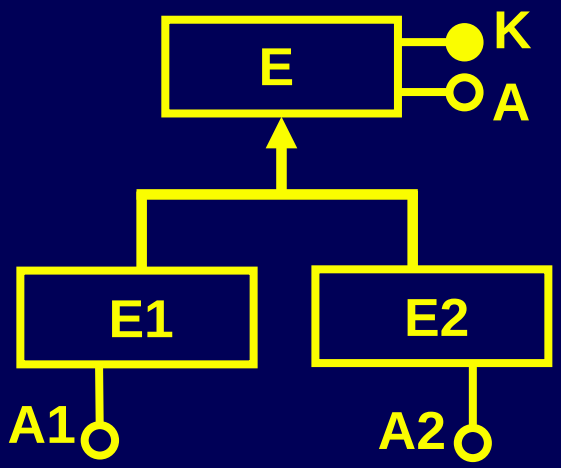
The translation takes place in 4 steps

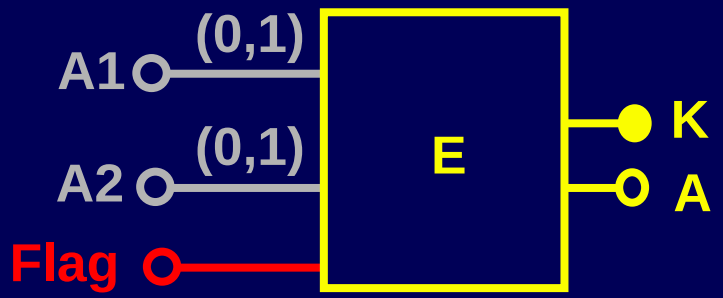
1. **Hierarchy elimination**
2. **Choice of primary keys**
3. **Normalization of multiple and composite attributes**
4. **Elimination (translation) of associations**

The output is a set of (relational**) TABLES**
with Primary Keys (mainly chosen among the identifiers of entities**)**
and Foreign Keys (expressing the references of associations**)**

1. Hierarchy elimination

- **Horizontal solution: one entity only**
  (corresponding to the «parent» entity)



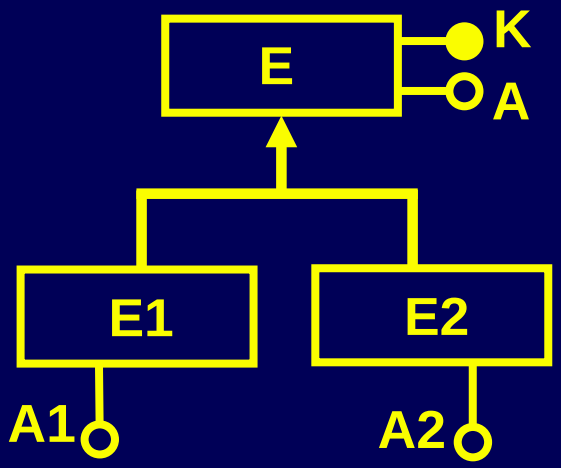


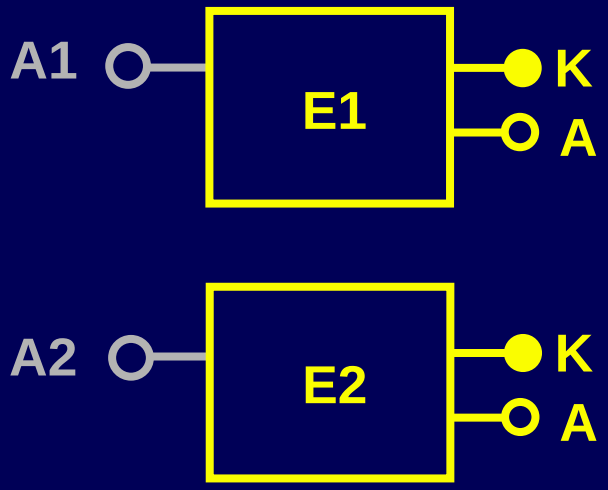
Flag: tells whether each instance is a E1 or a E2

Example: Person – Male / Female

1. Hierarchy elimination

- **Vertical solution: several entities (corresponding to «child» entities)**

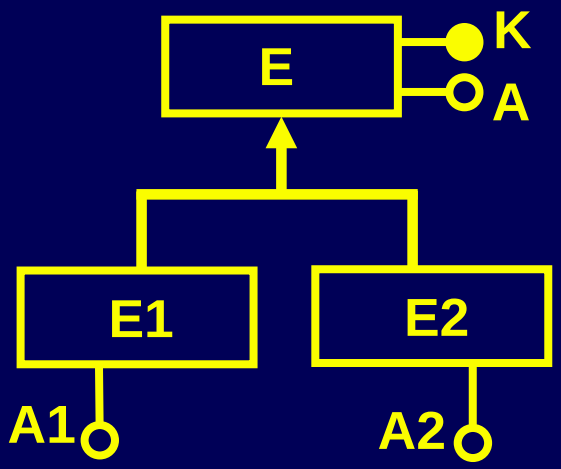


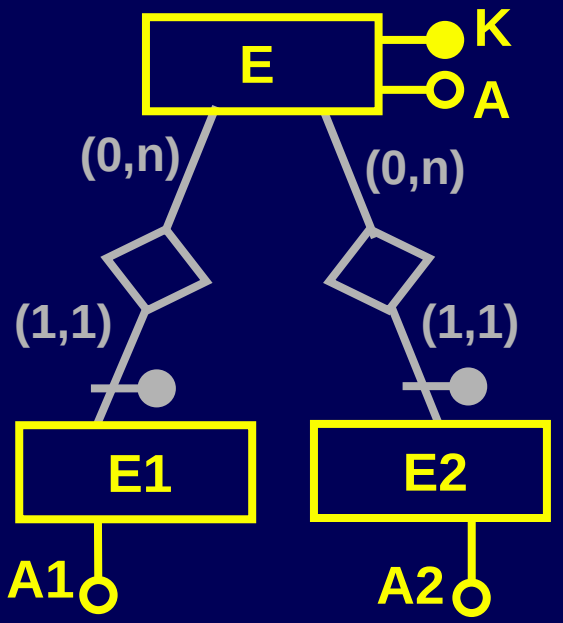


Example: Person - Student / Professor

1. Hierarchy elimination

- **Hybrid solution: keeping all the entities**
  (even if they are «very weak» entities)





Example: Person - Fisher / Hunter

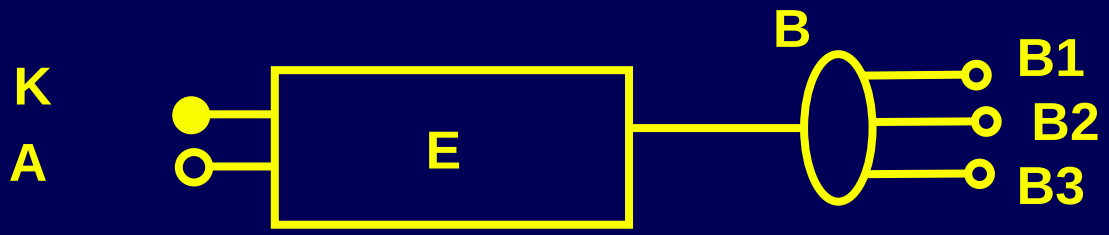
2. Choice of primary keys

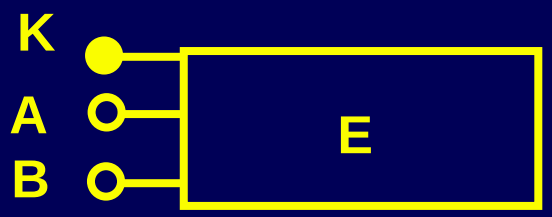
- Criterion: choice of the «most used» identifer, provided that it is simple (made of «few» attributes)
- If instead it is too complex, an ad-hoc code is introduced (typically a self-incrementing integer)
  - Numeric codes make joins more efficient w.r.t. (multiple) strings
- For weak entities, external identifiers must be “imported”
  - made «internal», so that the entity is artificially made «non-weak»

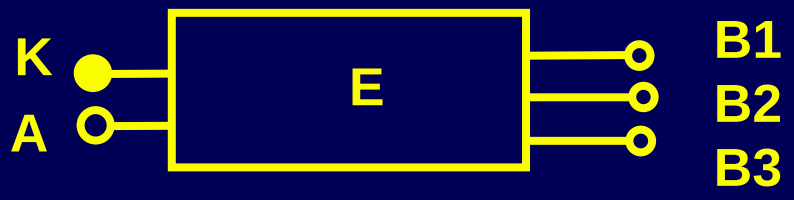
3.a Composite attributes

Must be reduced to simple attributes:

– either collapsing into one value (concatenation), or being split



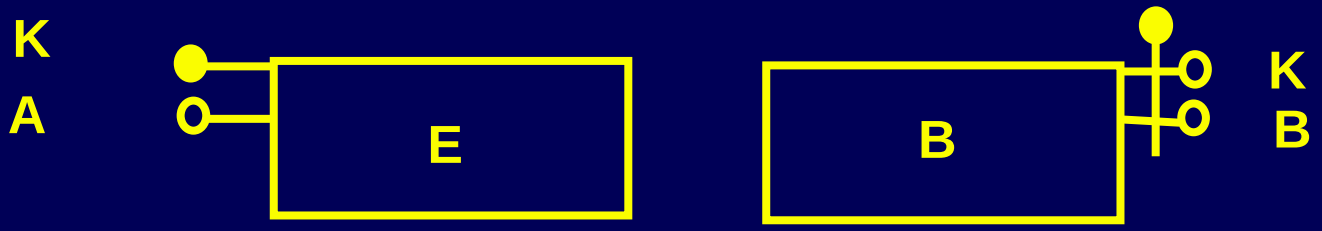




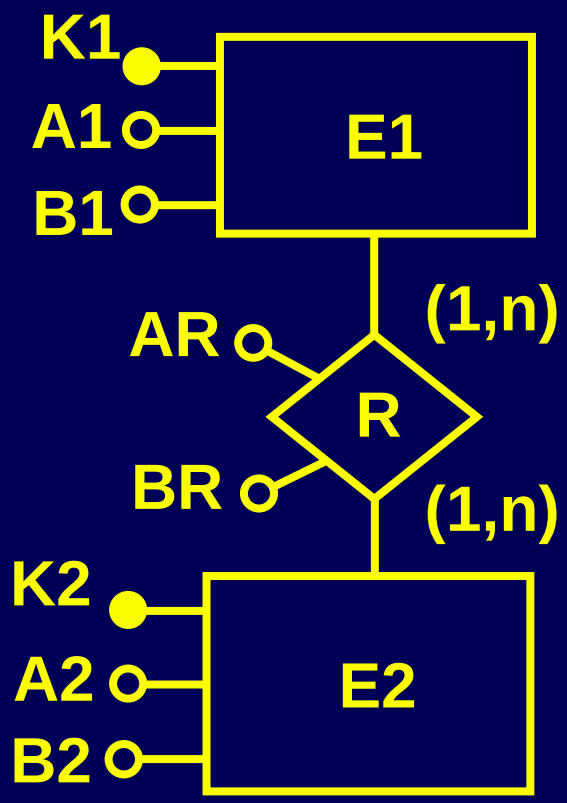
3.b Multiple attributes

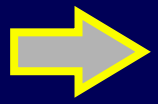
– Must be translated into a separate entity

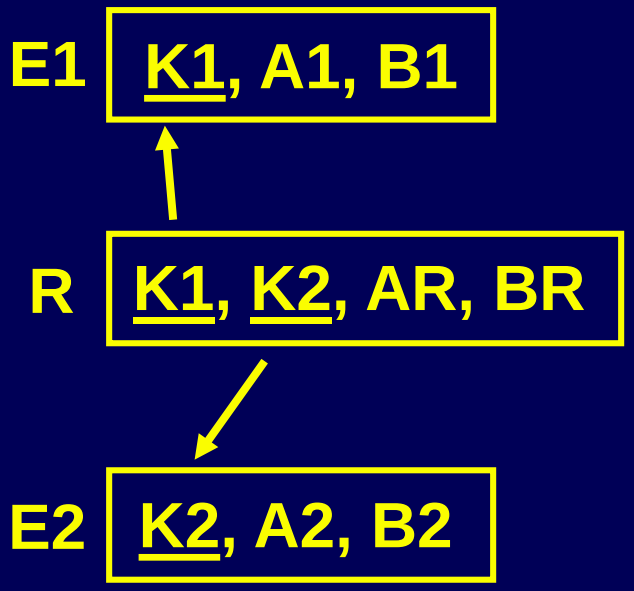




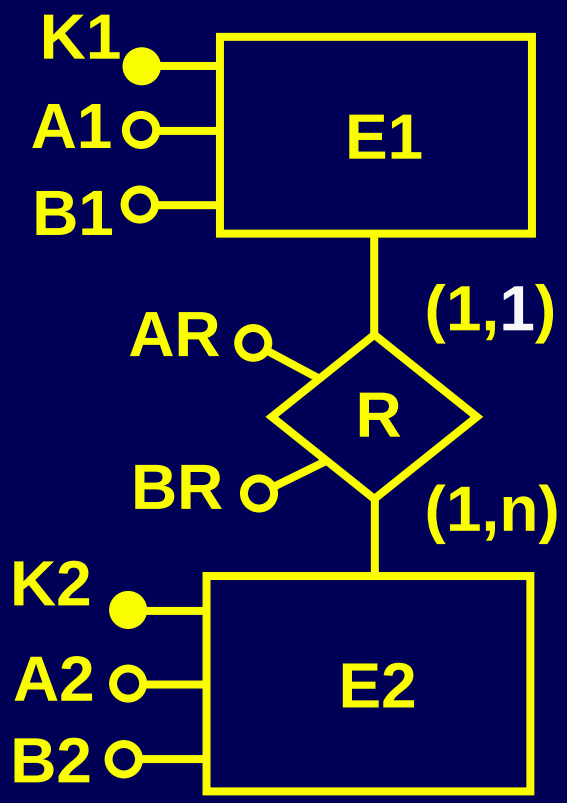
4. Translation of *many-to-many* associations

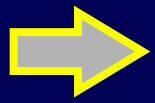


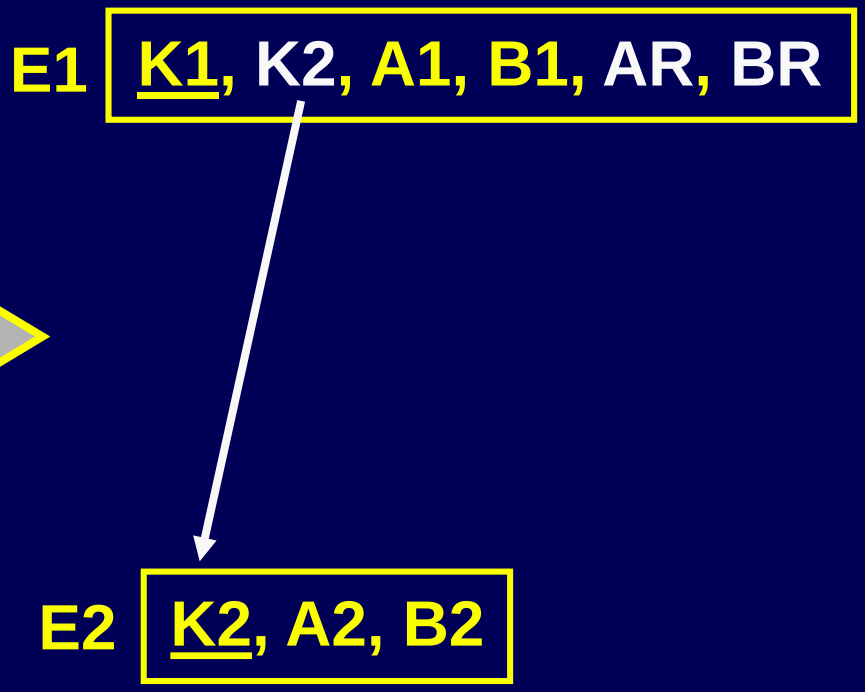




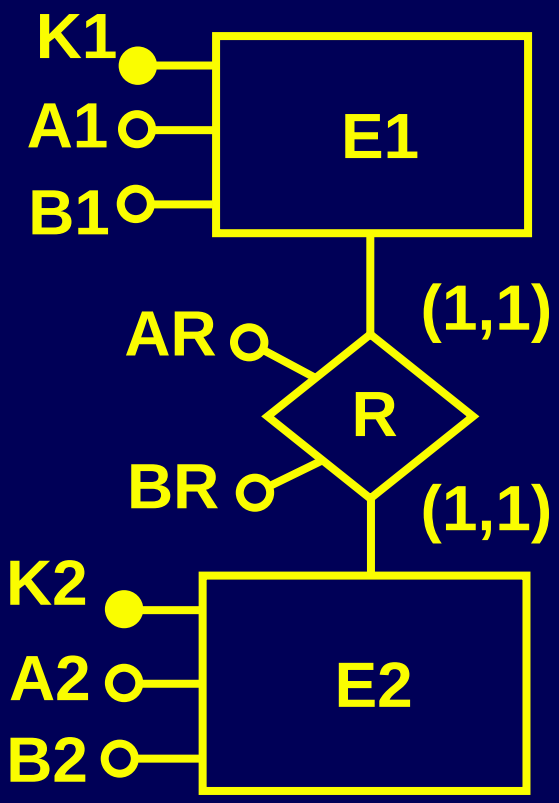
4. Translation of *one-to-many* associations







4. Translation of *one-to-one* associations (1/3)



E1 **K1**, A1, B1

R **K1**, K2, AR, BR

E2 **K2**, A2, B2

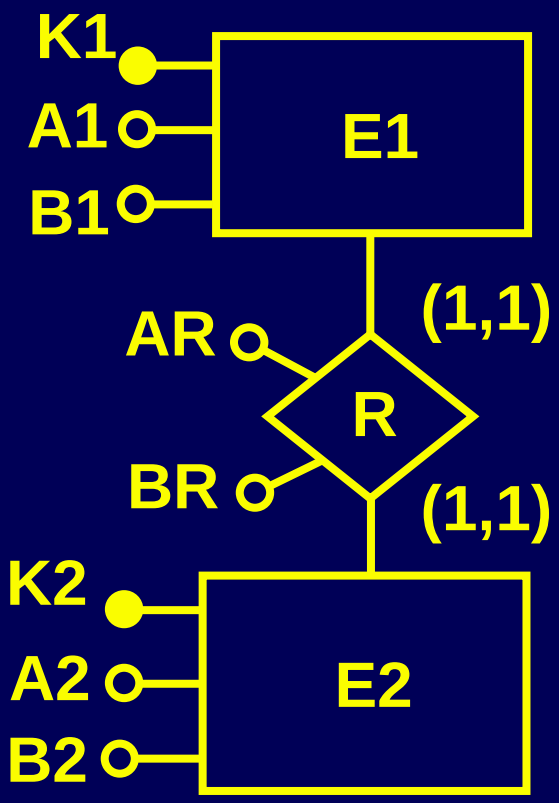
OR…

E1 **K1**, K2, A1, B1, AR, BR

E2 **K2**, A2, B2

OR…

4. Translation of *one-to-one* associations (2/3)



E1 **K1**, A1, B1

R K1, **K2**, AR, BR

E2 **K2**, A2, B2

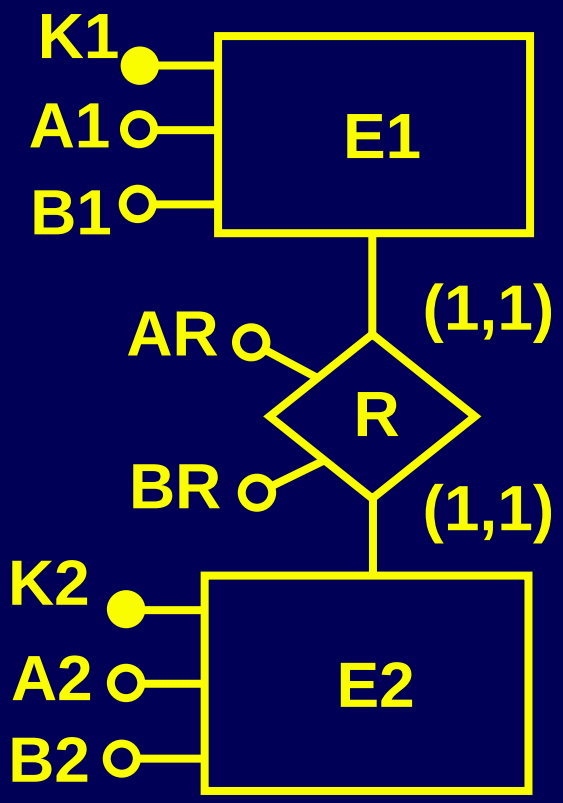
OR…

E1 **K1**, A1, B1

E2 **K2**, K1, A2, B2, AR, BR

OR…

4. Translation of *one-to-one* associations (3/3)



E | K1, K2, A1, A2, AR, BR, A2, B2

OR…

E | K1, K2, A1, A2, AR, BR, A2, B2